



Contents lists available at ScienceDirect

Information Sciences

journal homepage: www.elsevier.com/locate/ins

Concept learning for algorithmic reasoning: Insights from SAT-solving GNNs

Elad Shoham^{a,*}, Hadar Cohen^a, Khalil Wattad^a, Havana Rika^b,
Dan Vilenchik^a

^a School of Electrical and Computer Engineering, Ben-Gurion University of the Negev, Be'er Sheva, Israel

^b Information Systems, The Academic College of Tel Aviv-Yaffo, Tel Aviv-Yafo, Israel

HIGHLIGHTS

- We propose a framework for discovering algorithmic concepts learned by GNNs and showcase it on the problem of satisfiability using NEUROSAT by Selsam et al. (2018).
- Key abstractions, such as variable assignment confidence-level, emerge spontaneously when the GNN is trained to predict SAT/UNSAT using only single-bit supervision and standard cross-entropy loss.
- The discovered concepts enable both theoretical analysis and compression via a compact student network trained on internal representations.
- Our insights guide principled modifications to the classical WALKSAT algorithm, yielding new variants that converge faster.

ARTICLE INFO

Keywords:

Combinatorial optimization
Concept learning
Interpretability
Satisfiability
Graph neural networks

ABSTRACT

Explainable AI and model transparency methods primarily focus on classification tasks, identifying salient input features or abstract concepts that are directly tied to the data. In contrast, algorithmic problems such as SAT solving present a deeper challenge: here, meaningful concepts depend not only on the input but also on the model's evolving internal state; hence, such settings remain underexplored. We study concept learning in an existing model named NEUROSAT, a Graph Neural Network (GNN) trained to predict satisfiability, and uncover internal algorithmic structures, most notably the notion of *support*, that align with classical SAT heuristics. We then construct a significantly simplified GNN trained via a teacher–student approach: instead of learning from SAT/UNSAT labels, the student is trained to mimic NEUROSAT's latent representations—i.e., the concepts themselves—and achieves comparable performance using 91 % fewer parameters. For this simplified architecture, we provide a rigorous theoretical analysis that demonstrates, under certain assumptions on the input distribution and network weights, the emergence of the concept of support and its governing role in the network's dynamics. This work bridges explainability and algorithmic reasoning by showing that classical SAT-solving strategies emerge naturally in GNNs—and can be used to simplify, compress, and formally analyze their internal dynamics.

* Corresponding author.

Email addresses: shellad@post.bgu.ac.il (E. Shoham), vilenchi@bgu.ac.il (D. Vilenchik).

<https://doi.org/10.1016/j.ins.2025.122754>

Received 29 June 2025; Received in revised form 3 October 2025; Accepted 4 October 2025

Available online 9 October 2025

0020-0255/© 2025 The Author(s).

Published by Elsevier Inc.

This is an open access article under the CC BY license

(<http://creativecommons.org/licenses/by/4.0/>).

1. Introduction

AI systems are often viewed as efficient tools for tasks already within human reach. Yet, these systems may rely on unfamiliar concepts or hidden reasoning steps. Even when using known concepts, trust in AI increases when its reasoning is transparent and understandable. This motivates explainable AI (XAI), a field aiming to make model decisions interpretable.

A common XAI approach highlights input features that influence predictions—e.g., SHAP [1], LIME [2], GradCAM [3] — but these methods operate at the level of pre-defined and named per-sample features (e.g., in tabular data or pixel locations in an image), which are often hard to summarize meaningfully. In contrast, concept-based explanations identify human-aligned abstractions (e.g., “police logo” is an essential concept in identifying a police van) that generalize across inputs [4].

In this work, we focus on algorithmic tasks, where the goal is not classification but solving problems such as satisfiability (SAT) or graph coloring. For such tasks, Graph Neural Networks (GNNs) have emerged as the model of choice, owing to their ability to operate over relational structures and dynamically update latent states. Promising work in dynamic GNN research has focused on forecasting and perception—spatio-temporal models, such as DEST-GNN [5] for PV power prediction, SDR-GNN [6] for emotion recognition from multimodal sequences, and multi-scale architectures, including PMEDGN [7], for multivariate time-series modeling. These works highlight principles of evolving latent states, hierarchical organization, and distributed message flow that may also inform scalable concept extraction.

By contrast, our interest lies in algorithmic reasoning. GNNs have been applied to combinatorial problems through end-to-end training, including satisfiability [8] and graph coloring [9]. Other directions have explored emulation of classical algorithms [10], and recent work has even adapted GNNs to mimic semidefinite programming solvers through specialized loss functions [11].

Unfortunately, in all of these cases, the GNN models function as black boxes: their internal mechanisms cannot be directly understood or interpreted. This lack of transparency is critical because, as observed in our setting, GNNs often generalize to input graphs far larger than those seen during training. Yet the underlying reason for this generalization remains unclear, which makes it difficult to improve or fully trust the models’ behavior systematically. We suggest that concept learning may provide an explanatory mechanism for this phenomenon.

The difficulty in understanding how GNNs make decisions arises because, unlike in image classification, where a concept such as a “beak” is tied to a static region of the input, algorithmic tasks involve dynamic reasoning—concepts depend not only on the input graph but also on the GNN’s evolving hidden state over multiple iterations. Message-passing networks continually update node embeddings; thus, concepts need to capture changing dynamic quantities such as partial solutions or variable-level confidence, making mechanistic interpretation far less straightforward. Fig. 1 illustrates the dynamic versus static concept learning setting.

Before we proceed, it’s important to note that the notion of XAI typically aligns with *local, instance-level* explanation: attributing a specific decision to specific input features (e.g., saliency on pixels or concept activations), thereby answering “why this output for this input.” By contrast, our aim is *mechanistic, concept-level discovery*: identifying global algorithmic abstractions that govern a model’s internal state transitions across inputs and time. These perspectives are complementary. In spirit, our work is closer to approaches that visualize and analyze *internal dynamics* or extract *behavioral models*—such as visualizing agent memory over trajectories [12], learning finite-state abstractions of policies [13], and MDP-based explanation frameworks that reveal sub-goals or emergent structures [14]. We bring this mechanistic, concept-centric perspective to GNNs for combinatorial reasoning, formalizing the discovered concepts and linking them to both observed behavior and theory. Henceforth, when we use the terms *interpretation* or *explanation*, we refer specifically to this concept-level perspective, rather than to local instance-level attribution.

[15] provided an initial attempt at GNN dynamic concept learning: it inserts a supervised concept-bottleneck layer into a GNN that runs classical algorithms like BFS and Kruskal’s MST. With step-level labels for human-defined concepts (e.g., “has Visited Neighbour”), a shallow linear head maps these concepts to actions, allowing the learned weights to be read off as concise logical rules—demonstrating that concept-based explanations can extend beyond static, image-level phenomena to iterative graph algorithms.

The present work takes the next step by asking what algorithmic concepts emerge *spontaneously* when a GNN is trained end-to-end to solve a task, without any access to intermediate labels or handcrafted signals? As a test-bed, we study satisfiability (SAT) and analyze a pre-trained model named NEUROSAT by Selsam et al. [8], a message-passing GNN trained to predict SAT/UNSAT outcomes. By probing its internal dynamics, we extract a set of emergent algorithmic abstractions, most notably *support*, along with *assignment-consistency* and *backbone variables*, which we then translate into concise, human-interpretable rules that explain the solver’s decision-making process.

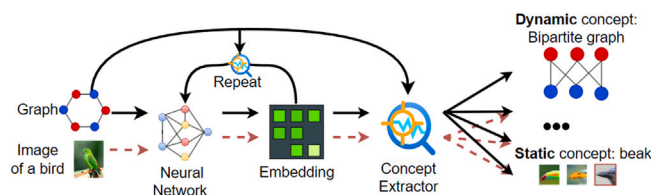


Fig. 1. Red dashed arrows illustrate the static concept learning pipeline: the input is embedded, and concepts (e.g., beak) are extracted from the embedding. Black arrows depict the dynamic setting, where both input and embedding evolve together through repeated NN application to solve an algorithmic task (e.g., bi-partiteness). Beak image adapted from [16].

Building on this foundation, we make four further contributions:

- (1) We show that the emergent concepts are encoded in a highly structured, low-dimensional subspace: over 95 % of the variance in the network’s internal representations is captured by the top two principal components, which encode the key abstractions such as support and assignment consistency.
- (2) We leverage this structure to train STUDENTNEUROSAT, a significantly smaller student GNN, using 91 % fewer parameters, guided solely by the teacher’s internal embeddings. This demonstrates that concept-level supervision enables both compression and behavioral transfer, even in complex algorithmic tasks.
- (3) We provide a theoretical analysis showing how the discovered concepts work together to drive NEUROSAT’s inference dynamics. We focus on the role of the *support* concept, which can be viewed as a proxy for the model’s confidence in a variable’s assignment. We prove that under certain assumptions, the support concept determines how variable embeddings evolve and how assignments stabilize, mirroring the variable assignment and re-assignment patterns observed experimentally.
- (4) We translate these insights back into algorithm design. By extracting NEUROSAT’s implicit flipping strategy, specifically, its preference for flipping low-support variables, we adapt the classical WALKSAT algorithm [17] with a modified flip rule. The resulting variant converges up to 1.5x times faster than standard WALKSAT versions, demonstrating how internal concept analysis can guide principled improvements to traditional heuristics.
- (5) Finally, we apply our pipeline to two additional GNNs designed for SAT and graph coloring. OPTGNN was trained in an unsupervised manner to solve MAX-SAT using a similar training set as NEUROSAT [11]. We find that support is not captured in its latent space representations, and correspondingly, the model fails to recover satisfying assignments. We also analyzed a GNN-GCP trained to predict graph 3-colorability, this time in a supervised manner with a similar architecture to NEUROSAT [9]. We observe the emergence of an analogous concept to the support in its latent space—a non-zero degree in two color classes. This concept structures the embedding space into a triangular geometry reflecting valid color classes.

The paper proceeds with a self-contained description of the SAT problem and relevant terminology in Section 2. In Section 3, we present a high-level overview of our theoretical results and demonstrate how they align with the patterns observed experimentally. Sections 4 and 5 survey related work and present the NEUROSAT model. Section 6 describes the data that we’ve used in this paper, and Section 7 details how the concepts manifest in the different datasets. In Section 8, we describe the simplified teacher–student version of NEUROSAT. In Section 9, we show how we updated WALKSAT using NEUROSAT’s conceptual insights to improve performance. Section 10 provides the rigorous analysis of NEUROSAT, Section 11 extends our analysis to two GNNs for SAT and graph colorability, and Section 12 concludes this paper with discussion and future work.

2. Preliminaries

We start with the definition of SAT and then review the key algorithmic concepts referred to throughout the paper.

Definition 1 (CNF Formula). A Boolean formula f is in k -CNF form if it is the conjunction (logical $\wedge$) of m clauses over n variables. Each clause has the form $(\ell_{i_1} \vee \ell_{i_2} \vee \dots \vee \ell_{i_k})$, where each literal ℓ_i is a variable x_i or its negation $\bar{x}_i$ (we focus on $k = 3$). To simplify notation, we index positive literals from 1 to n , and their negations from $n + 1$ to $2n$ (i.e., the negation of x_i is x_{n+i}).

The density of the formula is $c = m/n$, the ratio of clauses to variables. An assignment $\phi : \{\text{True}, \text{False}\}^n \rightarrow \{\text{True}, \text{False}\}$ satisfies f if every clause contains at least one literal evaluated to True under ϕ .

The Boolean Satisfiability (SAT) problem, first shown to be NP-complete by Karp [18], can be phrased both as a *decision* problem: does a given CNF formula admit a satisfying assignment? And as a *search* problem: find such an assignment if one exists. In this paper, we restrict attention to the canonical case of 3-SAT, where each clause contains exactly three literals.

Since SAT is NP-hard, researchers have long sought to understand its difficulty via *average-case analysis*. This has led to the study of various random SAT distributions, where instances are drawn from carefully defined ensembles. Random SAT has played a central role in theoretical and empirical investigations, and is also the primary focus of the original NEUROSAT work, as well as ours.

Definition 2 (Random and Planted SAT). An instance from the *Random 3SAT* distribution $\mathcal{R}(m, n)$ is sampled by randomly picking m clauses out of all $2^3 \binom{n}{3}$ possible clause configurations over n variables (First, 3 variables are chosen from the n variables, giving $\binom{n}{3}$ options, and then 2^3 ways to set the literal form of the 3 chosen variables, negated or not).

A *Planted 3SAT* instance in $\mathcal{P}(m, n)$ is generated by first picking an assignment ϕ and then picking m random clauses that are satisfied by ϕ .

Random SAT instances undergo a SAT-UNSAT phase transition as the density increases. The threshold is sharp as $n \rightarrow \infty$ and is located around $c = 4.26$ [19]. On the other hand, Planted SAT instances are satisfiable by definition for every c .

Definition 3 (the concept of support). We say that a variable x_i *supports* a clause $C = (\ell_i \vee \ell_j \vee \ell_h)$ with respect to an assignment ϕ if its literal ℓ_i evaluates to True and ℓ_j, ℓ_h evaluate to False. That is, x_i is the only one to satisfy that clause. The support count, denoted by $\text{supp}_\phi(x_i)$, is the number of clauses x_i supports with respect to an assignment ϕ .

For example, consider the clause $C = (x_j \vee \bar{x}_i \vee \bar{x}_h)$ and the assignment ϕ defined by:

$$\phi(x_j) = \text{False}, \quad \phi(x_i) = \text{False}, \quad \phi(x_h) = \text{True}.$$

Then under ϕ , the clause C is supported by x_i , and is evaluated as follows.

$$x_j = \text{False}, \quad \bar{x}_i = \text{True}, \quad \bar{x}_h = \text{False} \rightarrow \phi(C) = (\text{False} \vee \text{True} \vee \text{False}) = \text{True}.$$

For simplicity, from now on, when we refer to types of clauses with respect to an assignment ϕ , we use a *canonical form*. Specifically, we denote truth-value patterns by T (true) and F (false), and identify clauses up to permutation of their literals. Thus, the possible canonical forms are {TTT, TTF, TFF, FFF}, where, for example, TFF represents all of {TFF, FTF, FFT}, and TTF represents {TTF, TFT, FTT}.

It is worth noting that the notion of support is closely related to the classical concept of *unit clauses* in SAT solving, although it is not used directly in our analysis. Unit clauses arise during formula simplification steps, where a solver assigns a variable, removes satisfied clauses, and deletes falsified literals from remaining ones, forcing some clauses to contain a single unassigned literal. This mechanism, widely used in SAT-solving heuristics [17] reflects a similar intuition: when a clause relies heavily on a single variable for satisfaction, that variable's assignment becomes highly constrained.

3. Theory aligned with practice

In this section, we provide a high-level overview of the theoretical framework underpinning our results and illustrate how it manifests in the behavior of NEUROSAT. Our goal here is to convey the central intuitions and structural insights without getting bogged down in technicalities. We will return later to a more precise quantitative treatment in Section 7 and to fully rigorous proofs in Section 10. The discussion below is therefore deliberately informal, highlighting the key mechanisms that allow NEUROSAT to exploit specific structures of SAT formulas.

As we have mentioned above, support acts as a proxy for confidence: if many clauses rely solely on x_i to remain satisfied, then x_i 's current value is likely correct and should be preserved. Building on this intuition, we next give a high-level construction that uses support to define a combinatorial structure, the r -support core (or core for short), analogous to the graph-theoretic r -core. This theoretical construction, introduced in early works on Random Constraint Satisfaction Problems such as SAT [20] or graph coloring [21], and later extended in several variants [22], provides the foundation for our analysis. Using this framework, we will uncover the dynamics of NEUROSAT and connect its experimental behavior to a rigorous theoretical account.

3.1. Support-defined core (intuition)

It will be helpful for the reader to think of the core as a combinatorial structure with the property that, once the core variables are assigned correctly, the solver can *easily verify* their correctness: the assignment is stable, and the algorithm will not change it during search. In this sense, the core provides the solver with a form of confidence, anchoring the solution process.

In specific settings (to be discussed later), the notion of core coincides with the set of backbone variables or frozen variables—variables that take the same value in *all* satisfying assignments. In what follows, we will adhere to the core terminology and mention other relevant terms as needed.

In what follows, we refer to a fixed satisfying assignment ϕ of the formula. The core in SAT formulas is analogous to a graph r -core, a subgraph of minimum degree r . In SAT, clauses are heterogeneous hyper-edges that can be thought of as colored according to the number of satisfied literals with respect to ϕ (an hyper-edge is colored $g = 0, 1, 2, 3$ if the clause has g True literals). We define $\text{CORE}_r(\phi)$ as the set of variables that support at least r clauses, where the other two variables also belong to $\text{CORE}_r(\phi)$ (in other words, a subgraph where each vertex has at least r hyper-edges colored 1). For simplicity, we use CORE_r when ϕ is clear from context.

We now turn to explain why the definition of CORE_r focuses solely on support clauses and how they capture the essential source of confidence. Suppose that the algorithm has currently found an assignment ϕ' that is not satisfying but close to ϕ (say, they disagree on 10 % of the variables). Now consider a disagreement variable x_i which happens to belong to $\text{CORE}_r(\phi')$; that is $\phi'(x_i) = \text{True}$ but $\phi(x_i) = \text{False}$. Then each of its r supported clauses must contain at least one additional variable in $\text{CORE}_r(\phi')$ that is also set opposite to ϕ (otherwise the clause becomes FFF under ϕ which is not possible by the choice of it as a satisfying assignment). Propagating this argument, every clause supported by those new disagreement variables must also recruit another disagreement variable, and so on, yielding an “avalanche” of disagreements inside the core. However, when the core is large and the assignment ϕ' is already close to satisfying ϕ , the cascade scenario is impossible, and so $\text{CORE}_r(\phi')$ is correctly aligned with ϕ .

3.2. The NEUROSAT dynamics for the core

We now turn to the dynamics underlying NEUROSAT's operation and show how it is aligned with the notion of CORE_r just described.

At a high level, NEUROSAT implements message-passing cycles between literals. In contrast, clauses, where each literal or clause is represented by a high-dimensional embedding that evolves over iterations. For the sake of intuition, we will first describe these dynamics as if the embedding were a single scalar: support clauses act as positive evidence that push the literal scalar upward, while non-support clauses act as negative evidence that push it downward. This simplified one-dimensional description is not merely heuristic, as we will later see that projecting the high-dimensional embeddings onto their first principal component (PC1) indeed

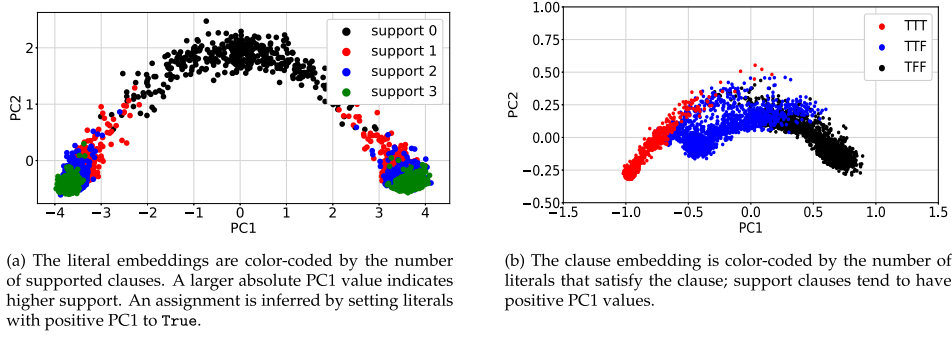


Fig. 2. PCA projection of literal and clause embeddings for a Random SAT instance with $n = 1500$ variables, and $m = 4.1 \cdot n$ clauses. In (a), literals are color-coded by the number of supported clauses and arranged symmetrically around $PC1 = 0$ due to literal-negation pairs. In (b), clause embeddings are color-coded by the number of satisfying literals; support clauses (type TFF) tend to have higher positive PC1 values.

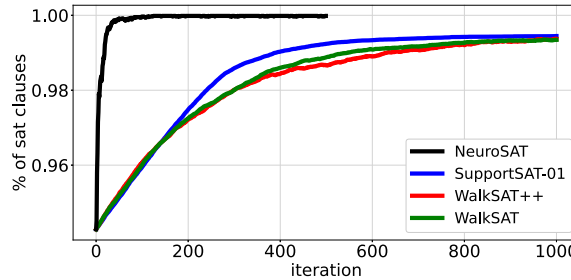


Fig. 3. The percentage of satisfied clauses is plotted against the iteration number. NEUROSAT satisfies almost all clauses after the first few iterations, and then slowly refines the assignment. Three versions of WALKSAT are plotted, which converge more slowly. The fastest-growing curve among the three is our “SUPPORTSAT-01”, which builds upon conceptual insights from NEUROSAT. Results are averaged over 100 instances with $n = 1500$ variables and $m = 4.1 \cdot n$ clauses.

yields a scalar score that captures the dominant signal in the dynamics. Under this view, literals with significant support are driven to higher absolute PC1 values, as illustrated in Fig. 2b and 2a.

Conversely, clauses update their embeddings by aggregating signals from their literals, which encode the literals’ current assignments. For intuition, consider a True literal as contributing +1 and a False literal as contributing −1. Fig. 2a illustrates the positive-negative breakdown. Under this view, support clauses accumulate a total of $(+1) + (-1) + (-1) = -1$, distinguishing them from other clause types that have positive values. When passed through the clause update layer of the network, this signal is flipped by learned weights, resulting in support clauses exhibiting the highest absolute PC1 values, which ties back to Fig. 2b.

The intuition extends further: if the two negatively contributing variables in a support clause themselves have significant support, then their embeddings carry stronger negative values, effectively replacing the −1 with a contribution of larger magnitude. After the clause update, this drives the clause’s PC1 even higher in absolute value. Iterating this effect across the formula, variables that co-occur with other high-support variables reinforce one another, pushing their embeddings consistently outward along PC1. This recursive amplification mechanism is how a CORE emerges in the embedding space—a cluster of variables that mutually reinforce each other.

The core argument becomes meaningful precisely when the current assignment is already close to a satisfying one. In this regime, high-support variables reinforce each other and resist being flipped, giving rise to the emergent core. Fig. 3 illustrates this dynamic. In the first few iterations, NEUROSAT rapidly gravitates toward a near-satisfying assignment, locking in the correct values for most variables. It then iteratively refines the residual disagreements—a process reminiscent of classical local search solvers, such as WALKSAT.

4. Related work

Concept Learning. Concept-based methods have become central to model interpretability by offering explanations in terms of human-aligned abstractions rather than low-level features [23]. These approaches typically define optimization programs over latent representations to extract concepts, enabling an intuitive understanding of the model’s message-passing dynamics. Research in this area spans supervised concept excavation using labeled data [24] and unsupervised discovery of the latent structure [4]. Other efforts integrate concept learning into dynamic settings, such as reinforcement learning (RL) [25].

The distinction between dynamic and static concepts can be further clarified by considering how internal representations evolve during computation. In contrast to static concepts—such as fixed visual features in image models [4]—dynamic concepts reflect internal quantities that change over time in response to an agent’s state and external feedback, while our concepts are purely from

the model's message passing internal state. For example, in RL, confidence in value estimates serves as a dynamic signal guiding exploration and adaptation [26]. In our setting, concepts like support and assignment consistency are not predefined but emerge, evolve, and adapt across message-passing iterations. This perspective aligns with recent frameworks in belief-driven abstraction [27], highlighting that dynamic concept learning is tightly linked to internal feedback, temporality, and structural adaptation.

The notion of concept-based cognition emphasizes the identification of essential concepts that support rule formation and decision-making [27]. Central to cognitive concept learning (CCL) is the dual notion of annotation (naming) and connotation (definition), along with statistical validation to establish conceptual significance. This aligns with the abstract-machine-brain framework proposed in CCL, where models are viewed as machines capable of abstract reasoning. Our work reflects this paradigm: concepts emerge without supervision or architectural modifications, suggesting that even task-driven neural networks can exhibit such abstract, interpretable structures.

Related lines of research include mechanistic interpretability, which reverse-engineers neural networks to uncover functional components [28], and recent efforts to extend concept learning to GNNs [29], as well as concept-based analyses of trained models such as [24], which discovered novel chess strategies in AlphaZero through embedding interpretation. Like these works, we study how concepts can emerge naturally without explicit supervision or architectural constraints. However, while these works focus on input patterns or static embeddings, we take it a step further by inspecting the evolving intermediate memory states to uncover how concepts are dynamically represented and utilized during computation. In this sense, our work, like [4], addresses *how* a dynamic concept affects the model's computation, rather than *why* the model assigns a particular concept to a given node.

Emerging trends include causal abstraction for revealing model mechanisms and human-in-the-loop systems that incorporate feedback on concept-level features [30]. Despite progress, challenges remain in ensuring completeness and generalizability [31]. Additional approaches, such as [15], require supervised concept labels and architectural bottlenecks to enforce interpretability.

Unlike prior work that relies on custom supervision signals, hand-crafted features, or architectural modifications to elicit interpretable computational dynamics, our approach demonstrates that meaningful algorithmic concepts, such as support, assignment consistency, and backbone variables, can emerge naturally from standard task-driven training. We show that even a generic GNN trained solely to predict SAT/UNSAT labels internally organizes its representations in a way that mirrors classical heuristics. This phenomenon not only deepens our understanding of how such models operate but also enables formal analysis and principled simplification of the architecture. Our results thus provide a new lens for studying learned algorithmic solver strategies in neural networks, beyond classification or perception tasks.

Although the concept learning literature primarily focuses on static domains, such as vision and text, our work shifts the focus to algorithmic reasoning tasks, where concepts are dynamic, internal, and state-dependent. This transition raises a natural question: Can such internal structures also emerge when neural networks are trained to solve classic combinatorial problems? To answer this, we ground our study in the domain of satisfiability (SAT), a well-studied benchmark in both theoretical CS and neural algorithm learning.

SAT Solving. We have already mentioned the classical hand-crafted SAT-solving heuristic WALKSAT [17]. Alongside such polynomial-time heuristics, exact solvers like zChaff [32] have been introduced and utilized in both academia and industry. In recent years, neural network-based heuristics have also been developed for various combinatorial optimization problems such as SAT.

[8] proposes an end-to-end NN-based approach called NEUROSAT. On the other hand, [33] incorporates NN into classical search algorithms to better guide the search. The SAT-solving community has explored the application of machine learning to SAT in more practical contexts. Possibly the most successful example is SATzilla [34].

GNNs have surfaced as a promising method for addressing SAT problems, providing viable alternatives to conventional backtracking and local search solvers [35]. Recent work has focused on combining GNNs with expert knowledge to improve SAT solver selection. For instance, [36] introduced NeuroComb, which enhances the Conflict-Driven Clause Learning (CDCL) algorithm using GNNs, demonstrating improved performance on recent SAT competition problem sets.

To address the lack of a unified evaluation framework for GNN-based SAT solvers, [35] presented G4SATBench. This comprehensive benchmark study evaluates various GNN models across different prediction tasks, training objectives, inference algorithms, and datasets (random, real-world, and cross-domain, such as clique and vertex cover). This benchmark offers insights into the performance of GNN-based SAT solvers, suggesting that existing GNN models can effectively learn a solving strategy akin to greedy local search but struggle to learn backtracking search in the latent space. Random instances are considered the most complex instances to solve, making them well-suited for our task of understanding how a model attempts to solve a SAT instance (as opposed to when it does).

Another approach, GraSS, has been proposed for automatic SAT solver selection based on tripartite graph representations of instances and a heterogeneous GNN model [37]. This method aims to overcome the limitations of existing SAT solver selection methods, which rely on hand-picked instance features that are costly to compute and overlook structural information in SAT graphs.

While NEUROSAT is not a state-of-the-art solver, as acknowledged by the authors of the original paper, we intentionally chose it as our subject of study. Despite its simplicity, uncovering and rigorously analyzing the internal concepts it learns remains a nontrivial task. Moreover, NEUROSAT generalizes well to larger problem instances, indicating that it internally represents some form of abstract, transferable concepts.

Our goal is not to advocate for NEUROSAT's competitiveness but to take a principled first step toward opening the black box of learned SAT solvers. We see this work as laying the methodological foundation for studying more advanced neural architectures and classical algorithms alike, with a clear path toward generalizing concept discovery and formal analysis in future work.

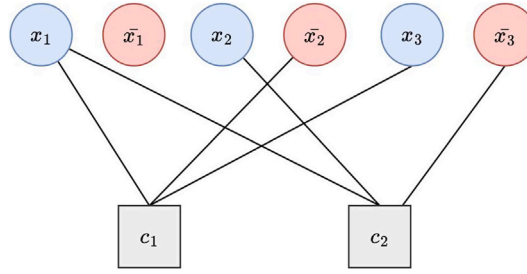


Fig. 4. The instance $f = (x_1 \vee \bar{x}_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \bar{x}_3)$ is transformed into an input graph for NEUROSAT where blue nodes are positive literals, red nodes are negated literals, and square nodes are clauses. A node is created for each clause and literal. Then, clause nodes are connected to the variables that appear in them.

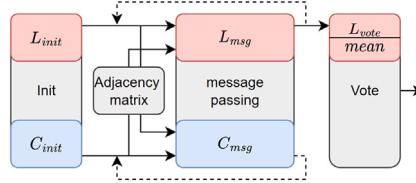


Fig. 5. A high-level view of the NEUROSAT architecture. NEUROSAT can be viewed as a message-passing algorithm, alternating between clause messages, C , and literal messages, L . This is done for t^{max} iterations. The update rules for iterations $t < t^{max}$ are $C^{t+1} \leftarrow C_u([C^t, ML_{msg}(L^t)])$ for the clauses, and $L^{t+1} \leftarrow L_u([L^t, FLIP(L^t), MC_{msg}(C^{t+1})])$ for the literals, where C_{msg}, L_{msg} are MLPs and C_u, L_u are LSTMs. At iteration t^{max} , compute $y \leftarrow \text{mean}(L_{vote}(L^{t^{max}}))$ to achieve the SAT/UNSAT prediction.

5. The NEUROSAT model

NEUROSAT [8] is a graph neural network that operates iteratively via message passing over a bipartite factor graph representing a CNF formula—clauses on one side, literals on the other, with edges connecting each literal to the clauses in which it appears (see Fig. 4).

The core architecture consists of two single-layer LSTMs, C_u and L_u , each with a hidden dimension of 128, which update clause and literal embeddings, respectively. These updates use two MLPs, C_{msg} and L_{msg} , and a *Flip* operator that swaps each literal with its negation. The clause and literal update rules are:

$$C^{t+1} \leftarrow C_u([C^t, M^T L_{msg}(L^t)]), \quad L^{t+1} \leftarrow L_u([L^t, Flip(L^t), MC_{msg}(C^{t+1})]), \quad (1)$$

where M is the adjacency matrix (and M^T is the transpose), L^{t+1} is a $2n \times 128$ matrix representing the literals' embeddings at iteration $t + 1$ and C^{t+1} is a $m \times 128$ matrix representing the clauses' embeddings at iteration $t + 1$. We use boldface to indicate model components (e.g., C_u, L_{msg}) to distinguish them from data tensors. Finally, NEUROSAT takes the literal embeddings at the final iteration t^{max} , applies an MLP L_{vote} , whose output for each literal is a “vote” on whether the instance is SAT/UNSAT. The votes are then averaged, $\text{mean}(L_{vote}(L^{t^{max}}))$, to obtain the final prediction.

NEUROSAT can thus be viewed as a message-passing algorithm alternating between clause-to-variable and variable-to-clause updates. The model architecture is detailed in Fig. 5. The network was trained using cross-entropy loss to predict satisfiability on millions of random CNF formulas with $n \in [10, 40]$, drawn near the SAT/UNSAT threshold (typically the density is $c \in [4, 6]$ for small n), with labels provided by an exact solver.

Beyond predicting satisfiability, NEUROSAT can also produce a candidate assignment by interpreting the final literal embeddings $L^{t^{max}}$. To do so, it applies k -means clustering to partition the embeddings into two groups corresponding to TRUE and FALSE. Each literal is labeled based on its proximity to the cluster centers. In cases where a literal and its negation fall into the same cluster, the one farther from the center is reassigned to the opposite cluster. This clustering procedure works effectively because of the symmetry visible in Fig. 6, where for most variables, the embeddings of a literal and its negation lie on opposite sides of the $x = 0$ axis.

6. Dataset

In the evaluation part, we focus on satisfiable instances, as our goal is to understand how NEUROSAT solves SAT, not how it refutes UNSAT formulas. To align with the original NEUROSAT setting, we use Random 3SAT instances for both training and analysis. This choice not only reflects the domain in which NEUROSAT was originally trained but also enables rigorous theoretical analysis, as random SAT is a well-established generative model in the algorithmic literature.

To probe generalization, we also include a second distribution: Planted SAT (Definition 2), which was not part of NEUROSAT's training. Planted instances enable us to generate much larger and denser formulas, extending well beyond the training regime of $n \in [10, 40]$ and the phase transition (defined by the c value), and explore how the learned concepts scale.

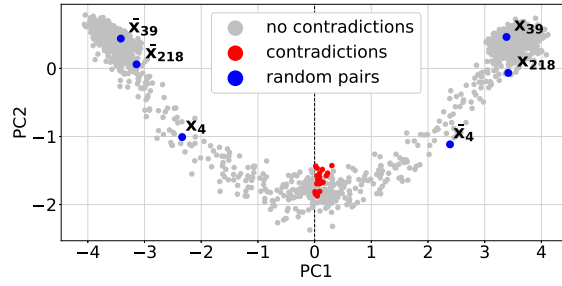


Fig. 6. PCA projection shows that literals and their negations are symmetrically positioned in the embedding space. Three random pairs of literals ($x_i, \bar{x}_i$) are colored with similar absolute PC2-values. Variables with contradictions are colored red and are all near PC1 equals 0.

Table 1

Datasets used for concept learning of NEUROSAT. For each combination of n and c , there are 100 random instances.

Dataset	# Instances	n	c	Source
SPARSE	2800	500, 1000, 1500, 2000	0.5, 0.8, 1, 1.25, 1.3, 1.5, 1.6	Random SAT
DENSE	2000	500, 1000, 1500, 2000	3.75, 4.0, 4.1, 4.2, 4.25	Random SAT
PLANTED	3500	500, 1000, 1500, 2000	from 4.5 to 15.5 jumps of 0.5	Planted SAT

Our datasets span a range of clause-to-variable ratios c , categorized as SPARSE and DENSE (all below the SAT-UNSAT threshold). These regimes are known to exhibit different combinatorial structures—e.g., phase transitions in the solution space [38]—suggesting that different concepts are useful in different density regimes. Table 1 summarizes the datasets we have used for the evaluation part.

7. Evaluation

In this section, we detail the concepts learned by NEUROSAT. All experiments were conducted on a personal machine equipped with an Nvidia RTX 3080 GPU, 64GB of RAM, and an AMD Ryzen 9 5650x CPU. The code can be found at <https://github.com/BGU-Elad/Concept-learning-for-algorithmic-reasoning-Insights-from-SAT-solving-GNNs>.

We ran NEUROSAT for at most 500 iterations per instance, significantly exceeding the 26 iterations used in the original paper, as we found this necessary for handling more challenging formulas. We extract an assignment every ten message-passing iterations using the approach from Section 5. If the formula is satisfied, we halt; otherwise, the iterations continue. Our analysis focuses on the first two principal components (PC1 and PC2), computed from the covariance matrices of the clause and literal embeddings C' and L' from Eq. (1).

Since each execution yields at most 500 covariance matrices per embedding type, and concepts reflect population-level patterns rather than instance-specific inference procedures, we average, per dataset, across all iterations and instances. For each dataset $D \in \{\text{SPARSE}, \text{DENSE}, \text{PLANTED}\}$, we compute an average covariance matrix S_D as follows. For a formula $f \in D$, let $S_{f,t}$ denote the covariance of clause (respectively, literal) embeddings at iteration t . The average covariance matrix S_D is given by

$$S_D = \frac{1}{|D|} \sum_{f \in D} \frac{\sum_{i=1}^{|S_f|} S_{f,i}}{|S_f|}$$

Finally, PC1 and PC2, the top two eigenvectors of S_D , capture approximately 98 % of the total variance (with PC1 explaining 90 % and PC2 8 %). These PCs can be viewed as distribution-level concepts for D , much like a “beak” might represent a semantic concept in a distribution of bird images.

We now turn to analyzing the key concepts learned by NEUROSAT. While our analysis uncovered several meaningful concepts, we focus here on the three most central: assignment consistency, support, and backbone variables.

7.1. The concept of assignment

Recall that assignments are extracted by clustering literal representations into TRUE and FALSE groups. Specifically, we apply k -means to the final literal embeddings and assign each literal based on its nearest cluster center. If both a literal and its negation fall into the same cluster, we have a contradiction, as a valid Boolean assignment must not assign the same truth value to both a variable and its negation. To resolve this, we identify the more distant literal from the pair and reassign it to the opposite group.

Fig. 6 shows the 2D PCA projection of NEUROSAT’s literal embeddings after 500 iterations, revealing a near-symmetric distribution around $x = 0$. Most literal pairs lie on opposite sides of the $x = 0$ vertical with similar y -values, inducing consistent assignments, with the positive PC1-score literal assigned to True.

Table 2 reports a low contradiction rate across datasets when assigning literals based on the sign of PC1. SPARSE instances have slightly higher rates due to literals appearing in only one polarity, leaving their negations unanchored. We resolve contradictions by assigning True to the literal with the higher PC1 value and False to its negation.

Table 2

High assignment consistency rates. The average % of contradictions per iteration over all iterations and instances in the dataset using PC1 values.

Dataset	% of contradictions by PC1
SPARSE	$7.3 \pm 2.7 \%$
DENSE	$3.2 \pm 1.5 \%$
PLANTED	$2.0 \pm 1.0 \%$

Table 3

Clause support concept. The average % of concept-abiding support clauses according to PC1 values.

Dataset	% of concept-abiding clauses by PC1
SPARSE	$61 \pm 8 \%$
DENSE	$89 \pm 6 \%$
PLANTED	$97 \pm 5 \%$

With the assignment concept in place, we next ask how often NEUROSAT's final assignment satisfies the formula. Our results align with the original paper [8] and with known results from the Random SAT literature: performance drops as the clause-to-variable ratio c approaches the SAT-UNSAT threshold. Details omitted.

7.2. The concept of support

We now illustrate how NEUROSAT computes clause embeddings from literal embeddings, and vice versa, tracing a complete cycle of message passing. For clarity, we describe this process using a simplified one-dimensional view, as if the embeddings were scalars. While this abstraction omits many architectural details, it captures the essential mechanism through which the concept of support (Definition 3) shapes the dynamics. As we walk through this narrative, we refer to empirical analyses (tables or figures) showing that the high-dimensional embeddings learned by NEUROSAT exhibit the same structural patterns predicted by this simplified perspective.

(1) Freeze clause embedding and compute literal embedding update: Each clause C has 0 to 3 literals satisfied under the current assignment ϕ_t . As shown in Fig. 2b, clause embeddings organize along PC1, with lower PC1 values corresponding to more satisfied literals. Clauses with exactly one satisfied literal (support clauses) tend to have positive PC1 scores; we refer to support clauses with a positive PC1 value as “concept-abiding.” Table 3 shows that the majority of support clauses are concept-abiding. The effect is most pronounced in DENSE instances, while SPARSE formulas exhibit lower rates. This is expected: each variable appears on average in $3c$ clauses, and on average only a $1/8$ -fraction, $3c/8$, are support clauses. When $c < 8/3$ (sparse regime), this expected count drops below 1.

Now for the literal embedding update. An instructive way to think of the update is as if a literal accumulates “evidence” from the clauses in which it appears as to how crucial the literal is to satisfy the clause. Support clauses (TFF) tend to have positive PC1 values, and non-support clauses tend to have negative ones. Hence, we can think of the LSTM L_u (Eq. 1, Literal part) as aggregating messages where each support clause contributes $+1$ and each non-support clause contributes -1 . The larger the sum, the more clauses “are asking” that literal to keep its current assignment, as it is the only one to satisfy that clause. Fig. 7a illustrates this variable update mechanism: literals with higher support have larger (absolute) PC1 values. We define the range $[-2, 2]$ as the zero-support zone (although it may contain a few non-zero support variables). As we'll see later, in iteration t , NEUROSAT reassigns only variables whose PC1 value lies in that zone in iteration $t - 1$. Table 4 provides more detailed statistics about the alignment between literal embedding PC1 values and the number of clauses a literal supports.

(2) Freeze literal embedding and compute clause update: keeping the one-dimensional illustrative perspective and simplifying the image in Fig. 6, let's suppose each literal embedding is $+1$ if it's positive and -1 if it's negative. Thus, a support clause (of type TFF) effectively aggregates $+1$ positive literal and -2 negatives, totaling in -1 . The other two types have embedding values 1 (TTF) and 3 (TTT). If the clause aggregation component C_u multiplies the literal summation by -1 , then we get back to our starting point – the image in Fig. 2 where more satisfying literals per clause mean a more negative PC1 value.

When the Narrative Breaks: Failed Executions. The support-based narrative outlined above sheds light on NEUROSAT's failure modes. Specifically, when the model fails to find a satisfying assignment, we observe a disruption in the encoding of support. We illustrate this through two types of failure cases: *close failures*, where the predicted assignment is within 1 % Hamming distance of a true solution, and *far failures*, where the distance exceeds 10 %. As shown in Fig. 7d, clause embeddings fail to isolate support from the TTF clauses. This breakdown leads to a weaker encoding of literal support (see Fig. 7b and 7c).

This degradation is also reflected in the last two rows of Table 4, where the alignment between PC1 range and true support weakens, especially in far-failed cases. Notably, literals with high support are often misrepresented as having low support, making them harder to assign correctly, potentially underpinning the network's failure to converge (see Section 9).

These failures align with the well-known shattering phenomenon in the solution space of random SAT formulas as the density approaches the SAT/UNSAT threshold. Shattering means that satisfying and near-satisfying assignments are split into isolated clusters [22]. Crucially, paths between these clusters traverse regions that violate a linear fraction of clauses—regions known as algorithmic

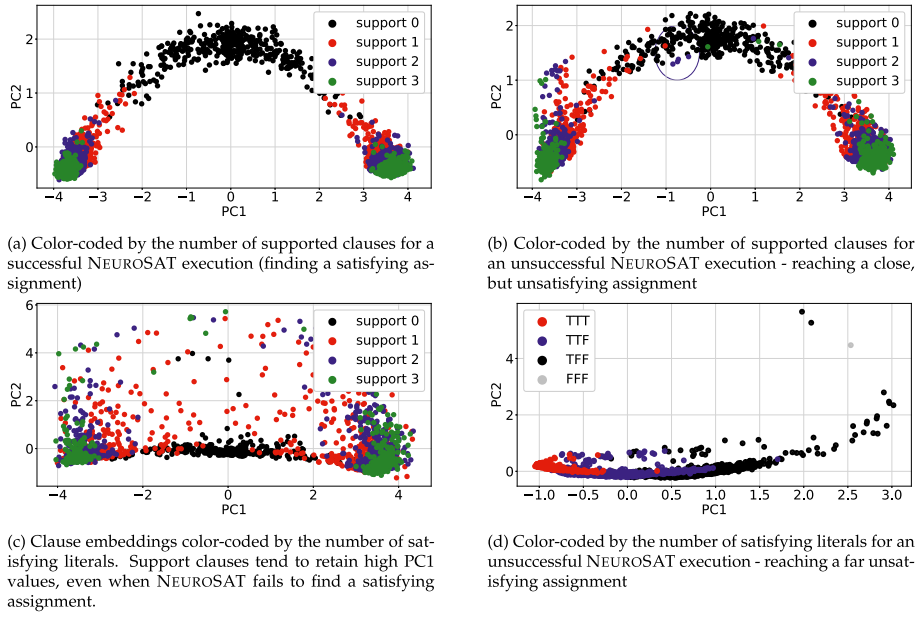


Fig. 7. PCA projection of the literals' and clauses' embedding for $n = 1500$ variables and $m = 4.1 \cdot n$ clauses. The embedding is color-coded with the support count. A larger absolute $PC1$ -value indicates greater support. In situations where NEUROSAT fails to find a satisfying assignment, support is encoded with noise.

Table 4

Literal support concept. $PC1$ range and the percentage of clauses with support s that fall in that range. We shorthand $\pm[\alpha, \beta]$ for $[-\beta, -\alpha] \cup [\alpha, \beta]$ on the x axis. The first two rows correspond to successful executions of Neuronsat. The next row corresponds to failed executions. The last row represents failed executions, and is far from every satisfying assignment.

Dataset	Support0	Support1	Support2	Support3
SPARSE	$\pm[0, 2.0]$ 94 %	$\pm[2.1, 3.3]$ 80 %	$\pm[2.7, 3.5]$ 86 %	$\pm[3.0, 3.7]$ 80 %
DENSE	$\pm[0, 1.5]$ 81 %	$\pm[1.6, 2.8]$ 76 %	$\pm[2.4, 3.0]$ 84 %	$\pm[2.6, 3.2]$ 92 %
DENSE FAILED	$\pm[0, 1.5]$ 60 %	$\pm[1.6, 2.8]$ 60 %	$\pm[2.4, 3.0]$ 70 %	$\pm[2.6, 3.2]$ 76 %
DENSE FAR	$\pm[0, 1.5]$ 46 %	$\pm[1.6, 2.8]$ 43 %	$\pm[2.4, 3.0]$ 54 %	$\pm[2.6, 3.2]$ 61 %

barriers. These barriers trap local search methods in nearly-satisfying basins, preventing them from reaching truly satisfying solutions. Since near-satisfying clusters vastly outnumber satisfying ones (by basic combinatorial reasoning), the model is statistically more likely to fall into one and get stuck, offering a plausible account for such failures in particular.

We also evaluated the model on out-of-distribution data from SATLIB [39], including morphed, flattened, and quasi-random formulas. Although the model fails to produce valid assignments on these instances, the internal representation of support remains visibly preserved. This suggests that the learned concept generalizes to some extent, even outside the training distribution. However, performance degrades significantly due to the prevalence of non-3CNF clauses—most formulas in these variants deviate from the strict 3CNF structure assumed both by the model and by our theoretical analysis (Section 10). As our primary focus is on 3CNF SAT, we do not explore these generalizations further.

7.3. The concept of core variables

The backbone of a CNF formula f refers to the variables that receive the same assignment in all satisfying solutions. For example, if x_1 and x_2 are in the backbone of f , then every satisfying assignment sets $x_1 = \text{True}$ and $x_2 = \text{False}$. These variables are sometimes referred to as frozen, but we will stick with the more common terminology: backbone. These variables appear to emerge as a higher-level construct built upon support.

In Section 3 we introduced the combinatorial structure CORE_r . By construction, the core has the property that if even a single core variable is misassigned with respect to some target satisfying assignment, an avalanche of disagreements is triggered: each violated clause forces further reassignments, propagating throughout the core. This dichotomy—either global consistency or large-scale correction—means that in regimes where all satisfying assignments lie close to one another in Hamming distance (i.e., form a tight cluster), the core induces a backbone of the formula. This is the case, for example, in Planted SAT instances with density c which have a unique backbone of size $(1 - e^{-\Omega(c)})n$. This result was already established in the literature, for example, in [40].

Random SAT formulas below the SAT/UNSAT threshold ($c \approx 4.26$) exhibit local backbones within clusters of solutions [22]. In both cases, the proofs rely on the notion of support.

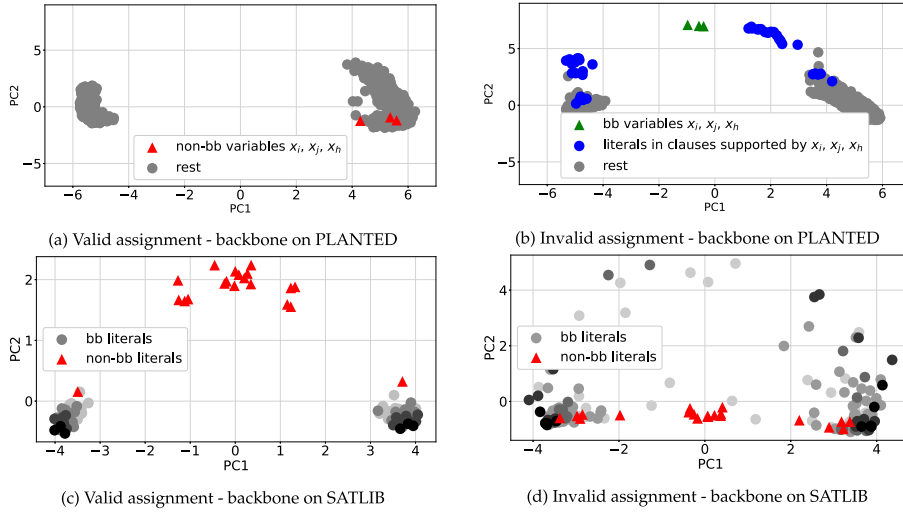


Fig. 8. Fig. 8a illustrates a clause C^* containing three random non-backbone variables x_i, x_j, x_h is added to a Planted SAT formula with $n = 1000$ variables and $m = 15 \cdot n$ clauses. Because x_i, x_j, x_h are non-backbone variables, the formula remains satisfiable, and no contradiction detection occurs. Fig. 8b is the same experiment, but this time x_i, x_j, x_h are part of the backbone. Making the instance unsatisfiable as x_i, x_j, x_h are all unsatisfied in C^* . NEUROSAT identifies the three by relocating their embedding to the support 0 zone. Fig. 8c is NEUROSAT's embedding of the backbone dataset ($b = 90\%$ from the SATLIB dataset). The black backbone variables are placed "correctly" outside the $[-2, 2]$ interval of zero support. Backbone variables are also color-coded by the number of supported clauses; the darker the greyscale, the higher the support. Fig. 8d shows the embedding when NEUROSAT fails to reach a satisfying assignment; in this case, some backbone variables are misplaced.

To probe NEUROSAT's handling of backbone variables for dense formulas, we generated PLANTED instances with $c = 15$ and the all-True Planted SAT assignment. We used the heuristic from [40] to identify three backbone variables x_i, x_j, x_h . Adding the clause $C^* = (\bar{x}_i \vee \bar{x}_j \vee \bar{x}_h)$, which contradicts the Planted SAT assignment, renders the formula UNSAT, despite C^* being just one of 15,001 clauses. The same clause using non-backbone variables leaves the formula SAT.

In the PLANTED dataset, each variable appears on average in $3c$ clauses, and $3c/7$ of these are expected to be support clauses. For $c = 15$, this yields an average support of 6–7 per variable, with low variance, resulting in no zero-support variables. This distribution is reflected in Fig. 8a, where the support encoding remains consistent and the "zero-support zone" (the $[-2, 2]$ PC1 range) is entirely free of variables, even when C^* is added, but when its variables are not backbone. In contrast, when C^* contains backbone variables, Fig. 8b shows NEUROSAT actively "suppressing" the support of C^* 's variables, as well as nearby literals, by relocating them into the $[-2, 2]$ region. This behavior was consistent across 100 randomly generated formulas with $n = 1000$ and clause densities $9 < c < 16$.

We also tested NEUROSAT on SATLIB backbone formulas [39]—RANDOM instances with backbone size $b \in \{30, 50, 70, 90\}$, $n = 100$, and $c \approx 4.25$. Across 800 instances per group, backbone variables lay outside the $[-2, 2]$ flip zone when NEUROSAT succeeded (Fig. 8c), and fell inside it during failures (Fig. 8d). Despite being out-of-distribution, these results demonstrate that NEUROSAT generalizes the support concept, reliably encoding backbone structure in successful runs.

The learned representation of NEUROSAT generalizes to complex structural properties such as backbone variables, even for distributions outside its training set. This indicates that NEUROSAT encodes high-level, global constraints, reinforcing the claim that its internal concepts are algorithmically meaningful, rather than dataset-specific artifacts.

8. Simplification and teachability

With a clearer understanding of the internal reasoning structures within NEUROSAT, we ask whether these concepts can be isolated, taught, and leveraged in a more efficient model. In other words: What is the minimal architecture that still thinks like NEUROSAT?

To answer it, we progressively simplified NEUROSAT's architecture and tested whether the resulting model could still be trained to maintain performance. Our findings reveal that even a highly simplified GNN, stripped of recurrence and nonlinear stacking, can replicate the key behaviors of NEUROSAT, provided it is trained via a concept-guided teacher–student approach.

Specifically, the clause update no longer depends on its history, and the literal update replaces concatenation with a subtraction of the literal and its negation (FC stands for a fully-connected layer; the remaining terms are defined in Section 5):

$$C^{t+1} \leftarrow \tanh(FC(M^T L^t)), \quad L^{t+1} \leftarrow \tanh(FC(MC^t) + FC(L^t - Flip(L^t))). \quad (2)$$

We refer to this model as STUDENTNEUROSAT. Unlike standard NEUROSAT, it cannot be trained via cross-entropy on satisfiability labels. However, we successfully trained it in a teacher–student setting, where the teacher is a trained NEUROSAT, and the student learns to match its internal representations.

To train STUDENTNEUROSAT, we first distilled the teacher's (NEUROSAT) representation space by adding an encoder–decoder of type fully-connected ($128 \rightarrow 64 \rightarrow 128$) to its clause and literal embeddings. These were trained after the teacher's training phase, producing a compact 64-dimensional representation. We then trained the student by minimizing the mean squared error (MSE)

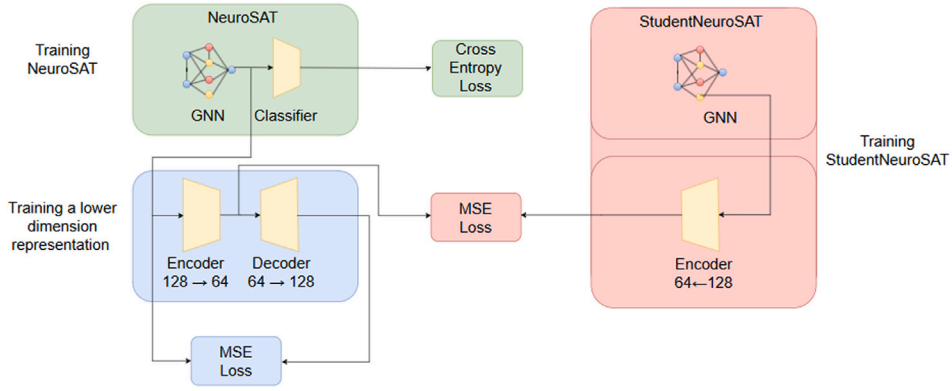


Fig. 9. Training losses for each component involved in training STUDENTNEUROSAT. Green shows the original NEUROSAT model trained with cross-entropy loss. Blue represents the encoder–decoder trained with MSE between the original and reconstructed embeddings, used to distill the teacher’s representation. Red shows the MSE loss used to train STUDENTNEUROSAT, comparing its embeddings to the teacher’s encoded embeddings.

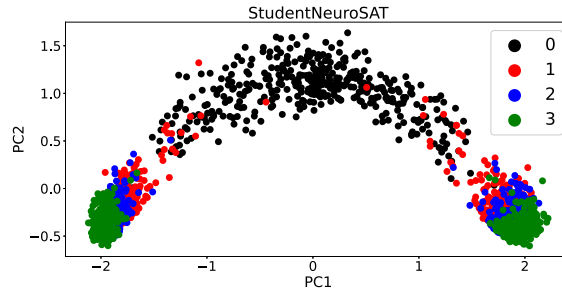


Fig. 10. Literals’ embeddings on STUDENTNEUROSAT. A general arch shape with the concept of support learned.

between this distilled teacher encoding and a linear projection of the student’s embeddings into the same 64-dimensional space. Training alternated between clause and literal alignment every 50 iterations. Formally:

$$\mathcal{L}_{\text{student}} = \frac{1}{n} \sum_{i=1}^n \|E_{\text{teacher}}(x_i) - P_{\text{student}}(x_i)\|_2^2$$

where E_{teacher} denotes the fixed 128→64 encoder trained on the teacher’s embeddings, and P_{student} is a learned linear projection from the student’s clause or literal embedding space to 64 dimensions. Notably, learning was only successful when using the reduced representation space—training directly on the full 128 dimensions failed to converge.

Our observation that STUDENTNEUROSAT could only be trained successfully when distilling the teacher’s 128-dimensional embeddings down to 64 dimensions aligns with findings in the knowledge distillation literature. Several works have shown that when the teacher’s representation is too large or complex relative to the student’s capacity, direct distillation often fails. [41] demonstrated that overly large teachers can produce knowledge the student cannot effectively mimic, and proposed Teacher Assistant Knowledge Distillation to bridge the capacity gap. Similarly, [42] found that high-dimensional intermediate features from the teacher could not be directly matched by a narrower student, requiring a learned projection to a smaller space. More recent work on embedding distillation [43] explicitly projects teacher embeddings into the student’s embedding dimension before comparison, ensuring learnability. Even [44] emphasized “softening” teacher outputs to simplify the signal. Collectively, these studies support the idea that reducing the dimensionality or complexity of the transferred knowledge is often necessary for successful student learning, which is consistent with our empirical finding that distillation in the original 128-dimensional space failed to converge.

Fig. 9 illustrates the training loss of STUDENTNEUROSAT w.r.t. NEUROSAT and the encoder–decoder.

The accuracy for STUDENTNEUROSAT is identical to that of NEUROSAT, and we replicated the results of Section 7. Fig. 10 portrays the concept of support using STUDENTNEUROSAT; the results resemble Fig. 7a.

Table 5 highlights the space and time advantages of using STUDENTNEUROSAT. The model achieves a 91 % reduction in both memory usage and parameter count. In terms of runtime, it finds solutions approximately 65 % faster. However, the number of iterations until a valid assignment is found is more variable. In about half the cases, STUDENTNEUROSAT converges in fewer iterations, while in the other half it requires significantly more. Despite this, the total time remains consistently lower.

Beyond its practical and interpretive advantages, the simplified update rule of STUDENTNEUROSAT also offers a key theoretical benefit: it enables a complete and rigorous analysis of the network’s operational characteristics under certain assumptions regarding the input and weights. The absence of recurrence and the linear structure of the updates make it possible to formally prove how the internal concept of support emerges and influences the dynamics. Details are in Section 10.

Table 5

Runtime and iteration statistics for STUDENTNEUROSAT and NEUROSAT across different datasets. The ratio column reports STUDENTNEUROSAT/NEUROSAT values as a fraction. Although STUDENTNEUROSAT runs for more iterations, it is still faster.

Metric	Dataset	StudentNeuroSAT	NeuroSAT	Ratio
Memory (MiB)	–	0.19	2.27	0.083
Params	–	50,306	594,690	0.084
	SPARSE	0.06 ± 0.07	0.17 ± 0.20	0.35
Time (sec)	DENSE	0.64 ± 0.42	1.85 ± 1.5	0.34
	PLANTED	0.21 ± 0.27	0.55 ± 0.77	0.38
	SPARSE	14 ± 25	13 ± 7	1.07
Iters	DENSE	295 ± 232	262 ± 264	1.12
	PLANTED	65 ± 141	50 ± 124	1.3

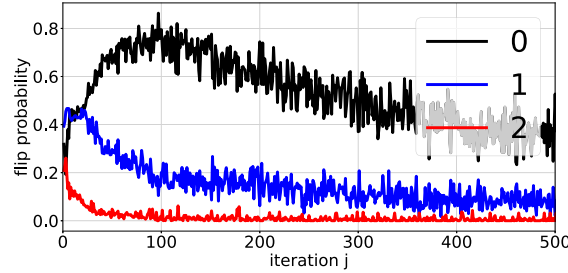


Fig. 11. The flip probability p_{si} for a variable with support $s = 0, 1, 2$ in iteration i ; p_s is inversely proportional to the support. This pattern, averaged over 100 runs with $n = 1500$ variables and $m = 4.1 \cdot n$ clauses, holds for all c 's in DENSE.

9. NEUROSAT as a WALKSAT algorithm

We design a new variant of the classical WALKSAT algorithm, inspired by the internal solver mechanism of NEUROSAT. Our perspective is to treat NEUROSAT as implicitly defining a probability distribution over variable flips, guided by the concept of support. By analyzing NEUROSAT's dynamics (see Fig. 11), we extract empirical flipping probabilities that prioritize low-support variables. In our variant, SUPPORTSAT-01, in every iteration, we first collect all variables appearing in currently unsatisfied clauses and partition them into two groups: those with support 0 and those with support ≥ 1 . We then select a variable to flip by choosing uniformly at random from the support 0 group with probability $2/3$, and from the support ≥ 1 group with probability $1/3$.

We chose to integrate NEUROSAT's extracted heuristics into a WalkSAT-style algorithm because WALKSAT and its variants are considered state-of-the-art within the algorithmic family of incomplete solvers—that is, solvers that do not rely on exact search techniques such as backtracking, branch-and-bound, or DPLL-style [45] methods. Given this, it is not surprising that neither NEUROSAT nor SUPPORTSAT-01 is competitive with exact solvers; instead, our goal is to improve local search-based methods by injecting learning-derived insights. For completeness, the basic structure of WALKSAT is provided below.

WALKSAT (Sketch)

Input: A CNF formula f over n variables and m clauses.

Initialization: Start from an assignment ϕ_0 (typically random or majority vote).

Iterative Step: Until timeout or solution:

- If the assignment ϕ_i at iteration i satisfies f , return it.
- With probability p , flip a random variable, and continue to the next iteration.
- Otherwise, pick an unsatisfied clause uniformly at random.
- Flip the assignment of a randomly chosen variable from that clause.

Output: Either a satisfying assignment or “UNSAT” if none is found in time.

We compare SUPPORTSAT-01 to two baselines: WALKSAT (with parameter $p = 0.1$, chosen via grid search) and WALKSAT + + [46], which flips support 0 variables if available, otherwise defaulting to a WALKSAT move.

Among the three WALKSAT variants, Fig. 12 shows that SUPPORTSAT-01 converges fastest, followed by WALKSAT + +, with standard WALKSAT being the slowest.

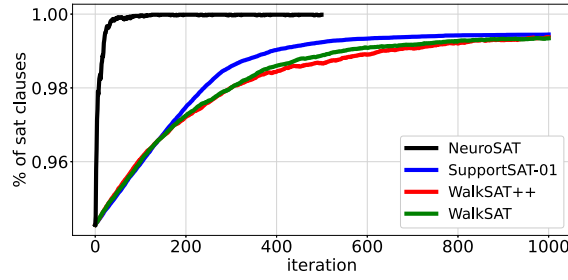


Fig. 12. SUPPORTSAT-01 improves WALKSAT. NEUROSAT converges the fastest, then SUPPORTSAT-01, WALKSAT++ and finally WALKSAT. All algorithms start from the majority vote. Averaged over 40 instances with $n = 1500$ variables and $m = 4.1 \cdot n$ clauses.

10. Rigorous analysis of NEUROSAT's dynamics

In this section, we formally state and prove a theorem that rigorously describes the dynamics outlined in Section 3. Like every rigorous proof, certain assumptions are made about the input and algorithm. In our case, we assume that the input is sampled from the Planted SAT distribution.

Although our proof is given for Planted SAT, the key takeaway is broader: we identify a combinatorial structure, the *support core*, that NEUROSAT can exploit. Planted SAT formulas provably contain this structure, making them a convenient setting for analysis.

Before stating the theorem and its proof, we recall the NEUROSAT simplified update rules and several key structural properties of Planted SAT formulas. We then introduce several definitions, including the distinction between *typical* and *atypical* variables: those whose local clause statistics match or deviate from their expected values under the Planted SAT model. The bulk of the proof focuses on analyzing the learned decision process of NEUROSAT when restricted to the set of typical variables. Finally, we demonstrate how to extend the argument to the complete formula, taking into account atypical variables. To streamline the presentation and focus on the key ideas, we defer some technical details to the appendix.

10.1. Update rules

Our analysis assumes an embedding dimension of $d = 1$, which may initially appear restrictive. However, this setting captures the essential update procedure of the network: empirical analysis shows that in Random SAT, over 90 % of the variance in NEUROSAT's embeddings is explained by PC1, and in Planted SAT, the explained variance exceeds 95 %. This confirms that the learned representation is effectively very low-dimensional.

Moreover, the key algorithmic concepts that emerge in NEUROSAT, *support* and *assignment consistency*, are themselves derived from a single scalar: the projection onto PC1. Thus, $d = 1$ is not merely sufficient for analysis; it directly reflects the dimensionality of the learned reasoning.

Recall the teacher-student update in Eq. (2):

$$C^{t+1} \leftarrow \tanh(FC_c(M^T L^t)), \quad L^{t+1} \leftarrow \tanh(FC_l(MC^t) + FC_\Delta(L^t - \text{Flip}(L^t)))$$

where we have a different fully-connected layer for the clauses (FC_c), literals (FC_l), and the difference between a literal and its negation (FC_Δ).

For $d = 1$, the fully-connected layers can be expressed as linear transformations:

$$FC_l(x) = a_l x + b_l, \quad FC_\Delta(x) = a_\Delta x + b_\Delta, \quad FC_c(x) = a_c x + b_c \quad (3)$$

10.2. Planted SAT facts

Planted model. Let f be a 3CNF formula generated from the Planted SAT distribution with parameter $c > 0$, where every clause that is satisfied by the Planted SAT assignment ϕ is included independently with probability

$$p = \frac{cn}{7 \binom{n}{3}} = \frac{6c}{7n^2} + o(1).$$

From now on, we drop the $O(1/n) = o(1)$ term to simplify the description. The expected number of clauses is $m = p \cdot 7 \binom{n}{3} = cn$. Without loss of generality, we assume the planted assignment ϕ is the all-true assignment. This simplifies presentation: e.g., instead of referring to the literal 'satisfied under ϕ ', we write x_i .

Clause counts. For any variable x_i let

$$X_i^+ = \#\{\text{clauses containing the positive literal } x_i\}$$

and similarly

$$X_i^- = \#\{\text{clauses containing the negative literal } \bar{x}_i\}.$$

Table 6

Expected number of each clause type. The $x_i\text{FF}$ cell for a negative literal is blank, as there are no FFF clauses.

Literals / Clause type	$x_i\text{TT}$	$x_i\text{TF}$	$x_i\text{FF}$	Total
Positive literal x_i	$3c/7$	$6c/7$	$3c/7$	$12c/7$
Negative literal $\bar{x}_i$	$3c/7$	$6c/7$	–	$9c/7$

Then, up to lower-order $O(\frac{c^2}{n})$ terms,

$$\mathbb{E}[X_i^+] = \binom{n}{2} \cdot 4p = \frac{12c}{7}, \quad \text{Var}(X_i^+) = \frac{12c}{7}, \quad (4)$$

$$\mathbb{E}[X_i^-] = \binom{n}{2} \cdot 3p = \frac{9c}{7}, \quad \text{Var}(X_i^-) = \frac{9c}{7}. \quad (5)$$

Clause subtypes. For a positive literal x_i , there are four possible ways to complete a clause with two literals so that this clause is satisfied by the planted assignment ϕ . The four evaluations are $x_i\text{TT}$, $x_i\text{TF}$, $x_i\text{FT}$, and $x_i\text{FF}$. For a negative literal $\bar{x}_i$ the $\bar{x}_i\text{FF}$ is not possible. Table 6 summarizes the expected number of each clause type, for positive and negative literals. The computation is straightforward, for example, the expected number of $x_i\text{TT}$ is $\frac{1}{4} \cdot \frac{12c}{7} = \frac{3c}{7}$.

Definition 4 (Typical variable). Fix the expected subtype counts from Table 6. A variable is called *typical* if, for every clause subtype, its observed count deviates from the expectation by at most $c/80$.

Lemma 1 (Prevalence of typical variables). For a random Planted SAT 3CNF formula with $m = cn$ clauses, Chernoff's inequality implies that

$$\Pr[a \text{ given variable is atypical}] = e^{-\Theta(c)},$$

so an $e^{-\Theta(c)}$ -fraction of literals (in expectation and with high probability) are atypical.

The lemma follows from a textbook application of Chernoff's bound, and its proof is omitted here. The proof can be found in [20], for example.

Definition 5 (Regular formula). A 3CNF formula f is regular if every variable it contains is typical.

When c is constant, a Planted SAT formula will not be regular with high probability. However, a larger sub-formula is. Recall the definition of $\text{CORE}_r(\phi)$, which is the set of variables that support at least r clauses, where the other two variables belong to $\text{CORE}_r(\phi)$ as well. We can extend this definition to require that the variables in $\text{CORE}(\phi)$ are all typical. For the proof, we need an additional condition: a core variable must appear in at most $c/20$ clauses with a non-core variable.

Proposition 1 (Size of $\text{CORE}(\phi)$). For a random Planted SAT 3CNF formula with $m = cn$ clauses, there exists a single $\text{CORE}(\phi)$ which consists of $(1 - e^{-\Theta(c)})n$ variables.

Proposition 1 was proven in prior work [47], which consists of standard probabilistic arguments.

Before we formally state our main theorem, we note that the initial literal embedding is fixed to the all-ones vector, which preserves symmetry and reflects the uninformative starting point used in practice. As motivated in Section 3, we assume $-a_c < 0$ (this somewhat nonconventional write-up is used to emphasize that the coefficient is *negative*), which ensures that the polarity of support clauses is correctly amplified in the network, and b_c is sufficiently large to ensure positivity after negative multiplication. For consistency with the choice $-a_c < 0$, we restrict all other scalar parameters to be positive.

The theorem is stated assuming the formula is regular. We then explain how to adjust the proof to accommodate irregularities, using the core and non-core decomposition.

Theorem 2. Given the NEUROSAT update rule in Eq. (3), and assuming that:

- (a) the embedding dimension satisfies $d = 1$,
- (b) the initial literal embedding is $L^0 = \mathbf{1}_{2n}$ (the vector of all ones),
- (c) the parameters satisfy $a_l, b_l, a_\Delta, b_\Delta, b_c > 0$ and $-a_c < 0$, and
- (d) b_c is a sufficiently large constant (its value will be made clear in the proof).

Then, when running on a regular 3CNF formula f with $m = cn$ clauses, it holds with high probability that:

- (i) (Support-Domination) For every iteration $t \geq 2$, the embedding of any TTT clause is smaller than the embedding of any TTF clause, which is smaller than any TFF clause.
- (ii) (Positive-Literal-Domination) For every iteration $t \geq 1$, the embedding of any positive literal is larger than the embedding of any negative literal.

(iii) For every iteration $t \geq 1$, all clause and literal embeddings are at least 0.75.

Proof. We prove the theorem by induction on the iteration counter t ; an iteration consists of both the clause update rule (first) and then the literal update rule.

Positive-Literal-Domination: base case ($t = 1$). At $t = 1$, the clause update rule in Eq. (2) computes the $m \times 1$ literal embedding vector C^1 :

$$C^1 = \tanh(-a_c(M^T L^0) + b_c) = \tanh(-a_c(3 \cdot 1_m) + b_c) = \gamma_0 \cdot 1_m > 0.75 \cdot 1_m.$$

where γ_0 is a constant determined by the value of $\tanh(-a_c(3 \cdot 1_m) + b_c)$, and thus depends on the parameters a_c and b_c . We can choose b_c sufficiently large so that $\gamma_0 \geq 0.75$. The second equality holds since every clause has exactly three literals, whose embedding is initialized to 1 at $t = 0$. The last inequality is true by assumption (d).

Next, the $2n \times 1$ literal embedding vector L^1 is computed from C^1 :

$$L^1 = \tanh(a_l(M C^1) + b_l + a_\Delta(L^0 - \text{Flip}(L^0)) + b_\Delta)$$

$L^0 - \text{Flip}(L^0)$ is zero because all literals have the same initialization at $t = 0$.

We use the notation $l_{x_i}^t$ to denote the row in L^t corresponding to the literal x_i (similarly $l_{\bar{x}_j}^t$). For the positive literal x_i , by the regularity condition on f , its embedding values $l_{x_i}^1$ are bounded by:

$$\tanh\left(a_l\left(\frac{12c}{7} + \frac{c}{20}\right)\gamma_0 + b_l\right) \geq l_{x_i}^1 \geq \tanh\left(a_l\left(\frac{12c}{7} - \frac{c}{20}\right)\gamma_0 + b_l\right), \quad (6)$$

and for a negative literal $\bar{x}_j$

$$\tanh\left(a_l\left(\frac{9c}{7} + \frac{c}{20}\right)\gamma_0 + b_l\right) \geq l_{\bar{x}_j}^1 \geq \tanh\left(a_l\left(\frac{9c}{7} - \frac{c}{20}\right)\gamma_0 + b_l\right). \quad (7)$$

The difference in the argument of the tanh between positive ($l_{x_i}^1$) and negative ($l_{\bar{x}_j}^1$) literal embeddings is $l_{x_i}^1 - l_{\bar{x}_j}^1 \geq a_l\left(\frac{3c}{7} - \frac{c}{10}\right)\gamma_0$, which is strictly positive, proving that $l_{x_i}^1 > l_{\bar{x}_j}^1$ (since tanh is monotonic). Furthermore, by increasing c , we can drive the argument of the tanh for both $l_{x_i}^1$ and $l_{\bar{x}_j}^1$ to be larger than 1, meaning $\tanh(1) > 0.75$ and proving $l_{x_i}^1 > l_{\bar{x}_j}^1 > 0.75$.

Support-Domination: base case ($t = 2$). From Eq. (3), at iteration $t + 1$:

$$C^{t+1} = \tanh(-a_c(M^T L^t) + b_c).$$

Repeating the argument for computing C^1 , row i in $M^T L^1$ equals the sum of the embedding values of the three literals in clause i . By the base case of the literal induction, the sum of three positive literals (TTT) is the largest, followed by TTF and finally TFF (with strict inequality). Since $-a_c$ is negative, this reverses the inequality direction. Adding a sufficiently large b_c completes the proof by ensuring a minimum value of 0.75 for clause embedding.

We have completed the proof of the base case. The induction step follows by similar arguments, though the analysis is more technical since we no longer assume regular initialization of the literal embeddings. The full proof is provided in Appendix A.

10.3. Proof extension for non-regular formulas

We now give more details on how to adjust the proof, taking into account the atypical variables. We start with the core variables and examine how the computation may be affected by the presence of clauses with non-core variables.

Let's revisit the proof and determine where it needs to be adjusted to accommodate non-core variables and the clauses in which they appear. For example, in Eqs. (6), (7) and (A.1), we need to add the contribution of $c/20$ clauses that may have any embedding value between 0 and 1. However, this affects the tanh by at most $c/20$, which can easily be accounted for by the slackness that we left in the constants we chose. This argument repeats in all computations, and all go through in the presence of non-core variables.

Item (iii) in Theorem 2 entails that from the second iteration onward, the assignment of the core variables doesn't change. This is because NEUROSAT picks the literal with the larger PC1 value and assigns it in a satisfying manner. Hence, from that iteration onward, assignment changes only for non-core variables, which we analyze next.

A key property in analyzing the assignment process of non-core variables is that when removing the core variables from the formula, the sub-formula induced by the non-core variables consists of disconnected components of size $O(\log n)$. This fact was previously proven in [20], and we omit the technical details here. From iteration $t = 2$ onward, updates occur only within this residual non-core graph. Even if NEUROSAT simply performed random flips over non-core variables, a satisfying assignment would be reached in time exponential in component size, still polynomial in n .

10.4. Clause weight structure in NEUROSAT

The central assumption in our proof regarding the GNN parameters is that the clause-update FC layer parameters satisfy $-a_c < 0$ and that b_c is sufficiently large, ensuring that the monotonic relationship among clause types is preserved.

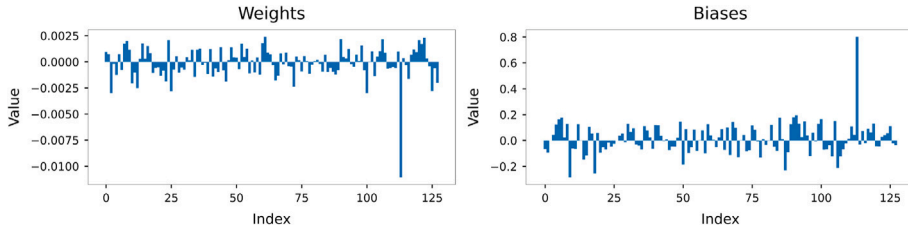


Fig. 13. Mean column weights and bias magnitudes for clauses, plotted by index. Opposing peaks appear at the same index, consistent with assumption (c) in Theorem 2.

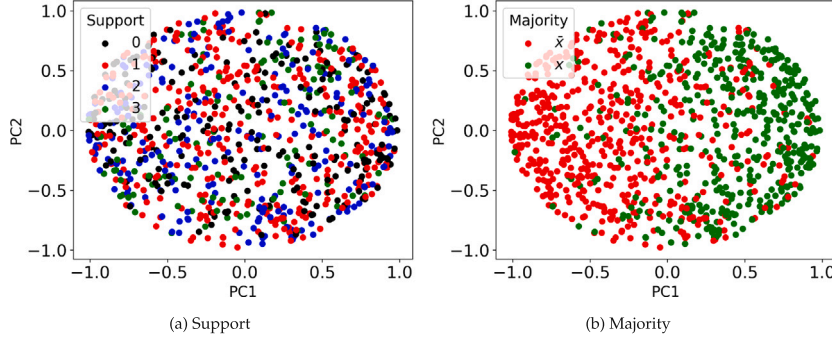


Fig. 14. 2D PCA projections of OPTGNN's embedding using the first two PC values. Points are color-coded by support in 14a and majority polarity in 14b (number of appearances as positive vs number of appearances as negated literal; green means more positive appearances and red means more negative). The input is a formula with $n = 1000$ variables and density $c = 3.5$. A clear correlation is observed for the majority (A positive PC1 value indicates that the variable is more likely to appear in its positive polarity form, while a negative PC1 value suggests the opposite). No consistent correlation is observed for the support. The same image repeats for any pairing of the first four PC values.

A direct comparison between our one-dimensional FC layer and the $d \times d$ layer in the update rule for C^{t+1} in Eq. (2) is not possible. To bridge this gap, we average the weights across each column, producing a one-dimensional summary of what each of the $d = 128$ embedding dimensions “feels.”

Fig. 13 shows these average column weights, alongside the $1 \times d$ bias vector. Strikingly, a single coordinate exhibits two opposing peaks—one in the weights and one in the bias—with the bias peak dominating the rest. This strongly supports our one-dimensional assumption, as it suggests that the effective learned structure is concentrated on a single dimension.

It is also noteworthy that NEUROSAT was not trained on Planted SAT, but rather on Random SAT formulas at much lower density. Thus, the fact that our theoretical assumption (derived in the Planted setting) still manifests in this regime underscores that the proof provides informative insights well beyond its original assumptions.

11. Other GNN solvers for SAT and coloring

We now apply our concept-discovery pipeline to two additional GNN architectures beyond NEUROSAT: one for SAT, and one for graph coloring. These case studies highlight how the emergence (or absence) of internal algorithmic concepts shapes performance. The SAT model we consider is OPTGNN [11], which differs from NEUROSAT in that it is trained in an *unsupervised* manner, using the semidefinite programming (SDP) loss for the MAX-SAT problem. The coloring model, GNN-GCP [9], is trained in a *supervised* manner on 2^{15} pairs of random 3-colorable and non-3-colorable graphs with 40–60 nodes, with edge density near the 3-colorability phase transition threshold (analogous to the setup in NEUROSAT).

We received the trained GNN-GCP from the authors of [9]. As for OPTGNN, we trained it ourselves according to the procedure described in the paper [11], replicating their results (a trained model is not publicly available).

Our findings show that OPTGNN fails to recover support-like structure and, in turn, never succeeds in producing satisfying assignments, whereas GNN-GCP learns a notion analogous to support (minimum degree by color class), and achieves some success on 3-coloring tasks. This contrast further supports our thesis that concept emergence provides a mechanism for understanding the generalization abilities of GNNs in algorithmic reasoning tasks. We now proceed with a detailed account of each GNN.

11.1. Case study: OPTGNN (SAT, unsupervised, negative example)

Yau et al. [11] propose OPTGNN, a general-purpose GNN designed to approximate solutions to semidefinite programs (SDPs), including relaxations of combinatorial problems like Max-Cut, SAT, and graph coloring. The model is trained in an unsupervised manner to minimize the SDP objective directly, leveraging message passing over the graph structure to encode constraints and capture feasible low-rank solutions. To extract a discrete assignment from the continuous embedding, the model employs rounding techniques (e.g., randomized projection or clustering methods such as k -means), reminiscent of Goemans–Williamson rounding.

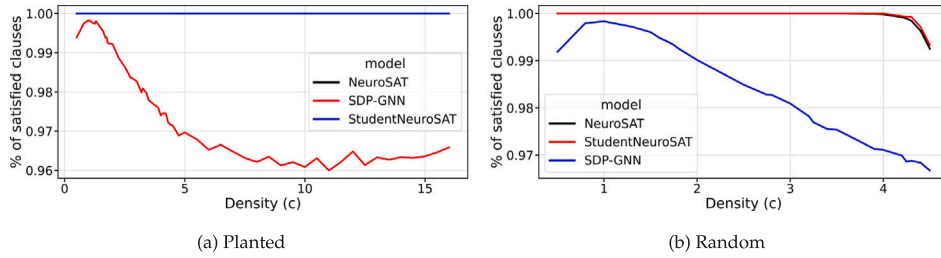


Fig. 15. The ratio of satisfied clauses (approximation ratio) for OPTGNN, NEUROSAT, and STUDENTNEUROSAT for the Planted SAT (left panel) and Random SAT (right panel) datasets on $n = 1000$ and different c values (averaged over 100 executions per c value). NEUROSAT and STUDENTNEUROSAT have almost identical performance, and achieve a better approximation ratio than OPTGNN, which rarely finds a satisfying assignment.

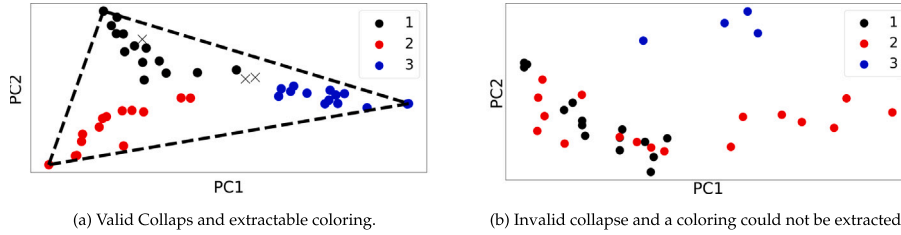


Fig. 16. 2D PCA projection of node embeddings for a planted 3-colorable graph with $n = 51$ nodes (classes of size 17 each). Inter-class edges are added with $p = 0.5$ (density ≈ 0.34). Nodes are color-coded by k -means clusters ($k = 3$). Successful runs yield a triangular projection with triangle vertices matching color classes; unsuccessful runs lack this separation. Three degree-8 nodes are marked 'x' with supports $4 + 4$, $7 + 1$, and $8 + 0$. The $4 + 4$ node lies nearest the black triangle vertex, illustrating the encoding of confidence by *support by color*—closer to triangle vertices means greater confidence.

Unlike NEUROSAT, where most information collapses into the first principal component, OPTGNN's representations remain higher-dimensional: four PCs are needed to explain 90 % of the variance. Moreover, the signal of *support*, which we have shown to be central in satisfiability, does not correlate with any component. Fig. 14a illustrates this lack of alignment, while Fig. 14b shows that the embeddings instead correlate with a simpler statistic: a variable majority vote (the number of positive appearances of x_i minus the number of negative appearances of $\bar{x}_i$).

In practice, OPTGNN was far less successful at recovering satisfying assignments than NEUROSAT or STUDENTNEUROSAT, as shown in Fig. 15. Taken together with the absence of a clear support signal in its embeddings, this provides suggestive evidence that the support concept is not only correlated with but also instrumental for successful SAT solving, at least in the Random SAT case.

11.2. Case study: GNN-GCP (Graph coloring, supervised, positive example)

Graph coloring presents a different testbed for concept emergence. Lemos et al. [9] introduced GNN-GCP, a supervised GNN with similar architecture to NEUROSAT, trained to classify whether a graph is k -colorable. Despite being trained only on decision outcomes, colorings can be extracted from its embeddings using clustering, much like assignment extraction in NEUROSAT.

In successful cases, we observed that GNN-GCP's embeddings are flattened into a distinct 2D triangular geometry (Fig. 16a), with each corner corresponding to a color class. Failed runs, by contrast, showed diffuse embeddings without geometric structure (Fig. 16b). Significantly, the strength of this triangular collapse correlated with a notion of *support by color*—the number of a node's neighbors in each of the other two classes. Nodes with higher support were pulled closer to their class's representative vector, mirroring the pull toward satisfying assignments in NEUROSAT. Fig. 16a illustrates this phenomenon: among three degree-8 nodes, the one with degree 4 in both the blue and the red color classes is embedded closest to the black corner, compared to two other degree-8 vertices that have $8 + 0$ and $7 + 1$ degree distributions in the other two classes.

12. Conclusion

This work extends concept learning to combinatorial optimization, focusing on SAT. Analyzing NEUROSAT, we uncover core algorithmic concepts, such as support, backbone, and assignment consistency, encoded in the principal components of its latent space. These concepts not only align with classical SAT heuristics but also underpin NEUROSAT's performance, including its generalization to out-of-distribution settings (Planted SAT and the backbone SATLIB instances).

Using our framework, we solve a longstanding puzzle surrounding NEUROSAT, noted by its original authors and echoed in multiple follow-up studies. NEUROSAT has a surprising generalization ability: the model is trained on very small SAT instances (e.g., 40–100 variables) but generalizes effectively to much larger ones (e.g., 1000+ variables). This behavior remained poorly understood, as no architectural mechanism explicitly supports such extrapolation. Our work provides a concrete narrative: NEUROSAT learns the concept of support that depends on clause density, not on the total number of variables. Since this concept remains meaningful across scales, the learned behavior naturally extends to larger instances.

We further show that these concepts are *teachable*: a significantly simplified GNN, trained via a teacher–student approach to mimic NEUROSAT’s internal representations (rather than SAT labels), replicates the same performance. This simplification is interesting in its own right, yielding a lightweight, interpretable model, but its true power lies in enabling formal analysis. For this minimal architecture, we provide a rigorous theoretical result proving the emergence and role of support. More broadly, our approach suggests a paradigm. By first extracting internal concepts from a complex model, one can design and analyze simpler student architectures that retain the essential reasoning patterns, opening the door to new forms of interpretability and mechanistic understanding in learned algorithmic systems.

A promising direction is to connect dynamic concept learning with the temporal reasoning capabilities developed in spatio-temporal GNNs, such as DEST-GNN [5]. While currently applied to physical forecasting, such models suggest mechanisms that might enhance long-range algorithmic planning in neural solvers. In addition, PMEDGN [7] emphasizes parallel multi-scale message-passing to model hierarchical temporal dependencies. This notion of layered latent structure resonates with our concept-based decomposition of NeuroSAT’s reasoning process and could inform future architectural simplifications or scaling strategies.

Another future direction is to complement our concept-discovery pipeline with *instance-level* explainability methods. While our work focuses on uncovering global algorithmic concepts such as support and backbone structures, future efforts could trace how these concepts drive specific predictions for individual SAT instances. For example, one could examine how the flip of a literal with low support propagates through the embedding space and ultimately influences the solver’s outcome, thereby linking concept dynamics to single-instance explanations. A first hint that such analyses are feasible appears in Table 4 and Section 11.1, where variations in support encoding are correlated with solver success. Extending these observations into a systematic framework would bridge global concept learning with local interpretability, offering a richer view of how GNNs reason in algorithmic domains.

In this work, concepts were extracted manually by computing principal components, reducing dimensionality, and leveraging domain expertise to identify key patterns in SAT-solving. While this approach demonstrates the feasibility of concept learning for algorithmic tasks, much of the process still relies on human interpretation. Techniques such as PCA, regression, and correlation analysis can be partially automated; however, assigning meaningful labels to learned structures, analogous to labeling regions in vision models, remains an inherently manual process. Developing automated pipelines for identifying, naming, and validating such algorithmic concepts is a promising direction for future research.

Another limitation of our work is that both the empirical insights and the theoretical analysis are grounded in Random and Planted SAT distributions. This modeling choice is common in the theoretical computer science community, where rigorous analysis typically relies on structural assumptions about the input distribution. However, when this methodology intersects with GNN-based learning, it can clash with the machine learning expectation of evaluating heterogeneous or real-world data. We acknowledge this gap: our results offer deep insight into NEUROSAT’s activation behavior under specific, idealized conditions, but do not yet extend to arbitrary or real-world SAT instances. We view this as a foundational step and hope that future work will build on these insights to relax the current assumptions and develop theoretical tools applicable to broader distributions.

CRedit authorship contribution statement

Elad Shoham: Writing – original draft, Visualization, Software, Methodology, Formal analysis, Conceptualization. **Hadar Cohen:** Software, Methodology. **Khalil Wattad:** Software, Methodology. **Havana Rika:** Writing – review & editing, Supervision. **Dan Vilenchik:** Writing – review & editing, Writing – original draft, Supervision, Conceptualization.

Declaration of generative AI and AI-assisted technologies in the writing process

Statement: During the preparation of this work, the author(s) used ChatGPT to edit sentences minimally. After using this tool/service, the author(s) reviewed and edited the content as needed and take full responsibility for the content of the publication.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Proof of Theorem 2–induction step

Literal-Domination: induction step. In what follows, we use the notation c_i^t for the embedding of clause i at iteration t . We further define $\bar{c}_{TTT}^t$ to be the maximum embedding value of a type- TTT clause in iteration t . Analogously, we define $\underline{c}_{TTT}^t$ for the minimum value. Now define $\delta_{TTT}^t = \bar{c}_{TTT}^t - \underline{c}_{TTT}^t$. Similarly, define $\delta_{TTF}^t, \delta_{TF}^t$. Finally, let $\delta^t = \max\{\delta_{TTT}^t, \delta_{TTF}^t, \delta_{TF}^t\}$.

Applying the literal update rule in Eq. (2) we compute the literal embedding L^{t+1} at iteration $t + 1$ using the clause and literal embeddings from iteration t :

$$\begin{aligned} L^{t+1} &= \tanh(a_l(MC^t) + b_l + a_\Delta(L^t - \text{Flip}(L^t)) + b_\Delta) = \\ &= \tanh\left(\underbrace{a_l(MC^t)}_{(1)} + \underbrace{a_\Delta(L^t - \text{Flip}(L^t))}_{(2)} + \underbrace{b_l + b_\Delta}_{(3)}\right) \end{aligned} \quad (\text{A.1})$$

We will bound each part (i.e., (1), (2), and (3) inside Eq. A.1) separately, for positive and negative literals. We denote by $(1)_{pos}$ the value in (1) for a positive literal, and similarly $(1)_{neg}$. By the regularity condition and the expectations in Table 6, we have that (1) for a positive literal x_i can be lower bounded by

$$(1)_{pos} \geq a_l \cdot \frac{12c}{7} \cdot \left(\frac{1}{4}c_{TTT}^t + \frac{1}{4}c_{TFF}^t + \frac{1}{2}c_{TFT}^t - \frac{1}{20} \right)$$

As for (2), since by the induction hypothesis, $l_{x_i}^t > l_{\bar{x}_i}^t$, we have

$$(2) = a_\Delta(l_{x_i}^t - l_{\bar{x}_i}^t) > 0$$

Similarly, for a negative literal $\bar{x}_j$,

$$(1)_{neg} \leq a_l \cdot \frac{9c}{7} \cdot \left(\frac{1}{3}c_{TTT}^t + \frac{2}{3}c_{TFT}^t + \frac{1}{20} \right),$$

and

$$(2) = a_\Delta(l_{\bar{x}_j}^t - l_{x_j}^t) < 0$$

Subtracting the value of $(1)_{neg}$ from $(1)_{pos}$, we get

$$(1)_{pos} - (1)_{neg} \geq a_l \cdot c \left(\frac{3}{7}c_{TFF}^t - \frac{9}{7}\delta^t - \frac{1}{10} \right). \quad (A.2)$$

Lemma 2 below asserts that $\delta^t \leq 0.1$, and together with the lower bound of 0.75 on c^t for every clause embedding, we obtain that

$$(1)_{pos} - (1)_{neg} \geq a_l \cdot c \cdot \frac{1}{15} > 0. \quad (A.3)$$

To complete this part of the proof, observe that (2) is positive for a positive literal and negative for a negative one (due to property (b) in the theorem), and that (3) is common to both. All in all, we have that $(1) + (2) + (3)$ is strictly greater for a positive literal than for a negative one.

Finally, we need to show that for all i , $\ell_{x_i}^{t+1}, \ell_{\bar{x}_i}^{t+1} \geq 0.75$. The worst case is that $\bar{x}_i$ appears in the least number of clauses, $\frac{9c}{7} - \frac{c}{20}$ (by the regularity condition and Table 6). Also, assume that each clause has the lowest possible value, 0.75. Then,

$$l_{\bar{x}_i}^{t+1} = \tanh \left[a_l \cdot \left(\frac{9c}{7} - \frac{c}{20} \right) \cdot (\tanh(-a_c \cdot 3 \cdot 0.75 + b_c) + b_l - a_\Delta(1 - 0.75) + b_\Delta) \right]$$

Choosing b_c and c sufficiently large, this simplifies to

$$l_{\bar{x}_i}^{t+1} \geq \tanh(a_l c + b_l - a_\Delta/4 + b_\Delta)$$

With $\text{arc tanh}(0.75) \approx 0.97$, it suffices to require

$$c \geq \frac{1 - b_l - b_\Delta + \frac{1}{4}a_\Delta}{a_l}.$$

Clause-Domination: induction step. To prove Clause-Domination, we use the same computation as in the base case, combined with the inductive claim on L^t , which completes the proof. $\square$

Lemma 2. Let $\delta^t = \max\{\delta_{TTT}^t, \delta_{TTF}^t, \delta_{TFF}^t\}$ be the maximum pairwise difference of clause-type embeddings at iteration t . Then, for a sufficiently large clause bias b_c ,

$$\delta_t < 0.01 \quad \text{for every } t \geq 2.$$

Proof (Lemma 2). Recall the clause update rule in Eqs. (2) and (3), and the lower bound (0.75) and upper bound (1) on every literal embedding; the difference between the largest and smallest c^t value is then.

$$|\tanh(-a_c(3 \cdot 0.75) + b_c) - \tanh(-a_c(3 \cdot 1) + b_c)|$$

By taking b_c sufficiently large, we can drive both elements of the difference as arbitrarily close to 1 as needed, so that their difference is smaller than 0.01. $\square$

Data availability

All data and code will be provided upon acceptance via github (and a link in the paper to reference github).

References

- [1] S.M. Lundberg, S.-I. Lee, A unified approach to interpreting model predictions, Adv. Neural Inf. Process. Syst. 30 (2017).
- [2] M.T. Ribeiro, S. Singh, C. Guestrin, "Why should I trust you?" explaining the predictions of any classifier, in: B. Krishnapuram, M. Shah, A.J. Smola, C.C. Aggarwal, D. Shen, R. Rastogi (Eds.), Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, ACM, San Francisco, CA, USA, 2016, pp. 1135–1144.

- [3] R.R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, D. Batra, Grad-cam: visual explanations from deep networks via gradient-based localization, in: *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 618–626.
- [4] A. Ghorbani, J. Wexler, J.Y. Zou, B. Kim, Towards automatic concept-based explanations, in: H.M. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E.B. Fox, R. Garnett (Eds.), *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8–14, 2019, Vancouver, BC, Canada, 2019*, pp. 9273–9282, <https://proceedings.neurips.cc/paper/2019/hash/77d2afcb31f6493c350fca61764efb9a-Abstract.html>
- [5] Y. Yang, Y. Liu, Y. Zhang, S. Shu, J. Zheng, DEST-GNN: a double-explored spatio-temporal graph neural network for multi-site intra-hour PV power forecasting, *Appl. Energy* 378 (2025) 124744, <https://doi.org/10.1016/j.apenergy.2024.124744>, <https://www.sciencedirect.com/science/article/pii/S0306261924021275>
- [6] F. Fu, W. Ai, F. Yang, Y. Shou, T. Meng, K. Li, SDR-GNN: spectral domain reconstruction graph neural network for incomplete multimodal learning in conversational emotion recognition, *Knowl. Based Syst.* 309 (2025) 112825, <https://doi.org/10.1016/j.knosys.2024.112825>, <https://doi.org/10.1016/j.knosys.2024.112825>
- [7] M. Hou, Z. Liu, G. Sa, Y. Wang, J. Sun, Z. Li, J. Tan, Parallel multi-scale dynamic graph neural network for multivariate time series forecasting, *Pattern Recognit.* 158 (2025) 111037, <https://doi.org/10.1016/j.patcog.2024.111037>, <https://www.sciencedirect.com/science/article/pii/S003132032400788X>
- [8] D. Selsam, M. Lamm, B. Büinz, P. Liang, L.D. Moura, D.L. Dill, Learning a SAT solver from single-bit supervision, *CoRR arXiv:1802.03685*, 2018. <http://arxiv.org/abs/1802.03685>
- [9] H. Lemos, M.O.R. Prates, P.H.C. Avelar, L.C. Lamb, Graph colouring meets deep learning: effective graph neural network models for combinatorial problems, *CoRR arXiv:1903.04598*, 2019. <http://arxiv.org/abs/1903.04598>
- [10] Q. Cappart, D. Chételat, E.B. Khalil, A. Lodi, C. Morris, P. Veličković, Combinatorial optimization and reasoning with graph neural networks, *J. Mach. Learn. Res.* 24 (130) (2023) 1–61.
- [11] M. Yau, N. Karalias, E. Lu, J. Xu, S. Jegelka, Are graph neural networks optimal approximation algorithms?, in: A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J.M. Tomczak, C. Zhang (Eds.), *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, December 10–15, 2024, Vancouver, BC, Canada, 2024*, http://papers.nips.cc/paper_files/paper/2024/hash/85db52cc08c5e0cfb1d216b1c85ba35-Abstract-Conference.html
- [12] T. Jaunet, R. Vuillemot, C. Wolf, DRLViz: understanding decisions and memory in deep reinforcement learning, *Comput. Graph. Forum* vol. 39 (2020) 49–61. Wiley Online Library.
- [13] A. Koul, S. Greydanus, A. Fern, Learning finite state representations of recurrent policy networks, *arXiv preprint arXiv:1811.12530*, 2018.
- [14] T. Zahavy, N. Ben-Zrihem, S. Mannor, Graying the black box: understanding dqns, in: *International Conference on Machine Learning*, PMLR, 2016, pp. 1899–1908.
- [15] D. Georgiev, P. Barbiero, D. Kazhdan, P. Veličković, P. Liò, Algorithmic concept-based explainable reasoning, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, 2022, pp. 6685–6693.
- [16] K. Cao, M. Brbic, J. Leskovec, 2020. Concept learners for generalizable few-shot learning, *CoRR arXiv:2007.07375*, <https://arxiv.org/abs/2007.07375>.
- [17] B. Selman, H.A. Kautz, B. Cohen, Local search strategies for satisfiability testing, in: D.S. Johnson, M.A. Trick (Eds.), *Cliques, Coloring, and Satisfiability*, *Proceedings of a DIMACS Workshop, October 11–13, 1993, Vol. 26 of DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, New Brunswick, New Jersey, USA, DIMACS/AMS, 1993, pp. 521–531.
- [18] R.M. Karp, Reducibility among combinatorial problems, in: R.E. Miller, J.W. Thatcher (Eds.), *Complexity of Computer Computations*, Plenum Press, New York, 1972, pp. 85–103.
- [19] A. Montanari, G. Parisi, F. Ricci-Tersenghi, Instability of one-step replica-symmetry-broken phase in satisfiability problems, *CoRR arXiv:cond-mat/0308147*, 2003. <http://arxiv.org/abs/cond-mat/0308147>.
- [20] A. Flaxman, A spectral technique for random satisfiable 3 CNF formulas, in: *SODA*, vol. 3, 2003, pp. 357–363.
- [21] N. Alon, N. Kahale, A spectral technique for coloring random 3-colorable graphs (preliminary version), in: *Proceedings of the Twenty-Sixth Annual ACM Symposium on Theory of Computing*, 1994, pp. 346–355.
- [22] D. Achlioptas, A. Coja-Oghlan, Algorithmic barriers from phase transitions, in: *49th Annual IEEE Symposium on Foundations of Computer Science*, IEEE, 2008, pp. 793–802.
- [23] M. Sundararajan, A. Taly, Q. Yan, Axiomatic attribution for deep networks, in: *International Conference on Machine Learning*, PMLR, 2017, pp. 3319–3328.
- [24] L. Schut, N. Tomasev, T. McGrath, D. Hassabis, U. Paquet, B. Kim, Bridging the human-AI knowledge gap: concept discovery and transfer in alphazero, *arXiv preprint arXiv:2310.16410*, 2023.
- [25] J. Shi, J. Xu, Y. Yao, B. Xu, Concept learning through deep reinforcement learning with memory-augmented neural networks, *Neural Netw.* 110 (2019) 47–54.
- [26] P. Li, M. Skripkin, A. Zubrey, A. Kuznetsov, I.V. Oseledets, Confidence is all you need: few-shot RL fine-tuning of language models, *CoRR arXiv:2506.06395*, 2025. <https://doi.org/10.48550/ARXIV.2506.06395>
- [27] D. Guo, W. Xu, W. Ding, Y. Yao, X. Wang, W. Pedrycz, Y. Qian, Concept-cognitive learning survey: mining and fusing knowledge from data, *Inf. Fusion* 109 (2024) 102426, <https://doi.org/10.1016/j.inffus.2024.102426>, <https://www.sciencedirect.com/science/article/pii/S1566253524002045>
- [28] N. Nanda, L. Chan, T. Liberum, J. Smith, J. Steinhardt, Progress measures for grokking via mechanistic interpretability, *arXiv preprint arXiv:2301.05217*, 2023.
- [29] H. Sun, Y. Li, Y. Zhang, Conlearn: Contextual-knowledge-aware concept prerequisite relation learning with graph neural network, in: A. Banerjee, Z. Zhou, E.E. Papalexakis, M. Riondato (Eds.), *Proceedings of the 2022 SIAM International Conference on Data Mining, SDM 2022, April 28–30, 2022, Alexandria, VA, USA, SIAM*, 2022, pp. 118–126, <https://doi.org/10.1137/1.9781611977172.14>
- [30] J. Hong, K.H. Park, T.P. Pavlic, Concept-centric transformers: enhancing model interpretability through object-centric concept learning within a shared global workspace, in: *IEEE/CVF Winter Conference on Applications of Computer Vision, WACV 2024, January 3–8, 2024, Waikoloa, HI, USA, IEEE*, 2024, pp. 4868–4879, <https://doi.org/10.1109/WACV57701.2024.00481>
- [31] Z.C. Lipton, The mythos of model interpretability, *Commun. ACM* 61 (10) (2018) 36–43, <https://doi.org/10.1145/3233231>
- [32] Y.S. Mahajan, Z. Fu, S. Malik, Zchaff2004: an efficient SAT solver, in: *Theory and Applications of Satisfiability Testing: 7th International Conference, SAT 2004, May 10–13, 2004, Revised Selected Papers 7, Vancouver, BC, Canada, Springer*, 2005, pp. 360–375.
- [33] D. Selsam, N. Björner, Guiding high-performance SAT solvers with UNSAT-core predictions, in: *Theory and Applications of Satisfiability Testing–SAT 2019: 22nd International Conference, SAT 2019, July 9–12, 2019, Proceedings 22, Lisbon, Portugal, Springer*, 2019, pp. 336–353.
- [34] L. Xu, F. Hutter, H.H. Hoos, K. Leyton-Brown, SATzilla: portfolio-based algorithm selection for SAT, *CoRR arXiv:1111.2249*, 2011.
- [35] Z. Li, J. Guo, X. Si, G4SATBench: benchmarking and advancing SAT solving with graph neural networks, *CoRR arXiv:2309.16941*, 2023. <https://doi.org/10.48550/ARXIV.2309.16941>
- [36] W. Wang, Y. Hu, M. Tiwari, S. Khurshid, K.L. McMillan, R. Miikkulainen, NeuroComb: improving SAT solving with graph neural networks, *CoRR arXiv:2110.14053*, 2021. <https://arxiv.org/abs/2110.14053>
- [37] Z. Zhang, D. Chételat, J. Cotmareanu, A. Ghose, W. Xiao, H. Zhen, Y. Zhang, J. Hao, M. Coates, M. Yuan, Grass: combining graph neural networks with expert knowledge for SAT solver selection, in: R. Baeza-Yates, F. Bonchi (Eds.), *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2024, August 25–29, 2024, Barcelona, Spain, ACM*, 2024, pp. 6301–6311, <https://doi.org/10.1145/3637528.3671627>
- [38] S. Mertens, M. Mézard, R. Zecchina, Threshold values of random K-SAT from the cavity method, *Random Struct. Algorithms* 28 (3) (2006) 340–373.
- [39] H.H. Hoos, T. Stützle, SATLIB: an online resource for research on SAT, *SAT 2000* (2000) 283–292.
- [40] A. Coja-Oghlan, M. Krivelevich, D. Vilenchik, Why almost all satisfiable k -CNF formulas are easy, in: *Discrete Mathematics and Theoretical Computer Science*, *Discrete Mathematics and Theoretical Computer Science*, 2007, pp. 95–108.
- [41] S. Mirzadeh, M. Farajtabar, A. Li, N. Levine, A. Matsukawa, H. Ghasemzadeh, Improved knowledge distillation via teacher assistant, in: *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, the Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, the Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, February 7–12, 2020, New York, NY, USA, AAAI Press*, 2020, pp. 5191–5198, <https://doi.org/10.1609/AAAI.V34I04.5963>

- [42] A. Romero, N. Ballas, S.E. Kahou, A. Chassang, C. Gatta, Y. Bengio, Fitnets: hints for thin deep nets, in: Y. Bengio, Y. LeCun (Eds.), 3rd International Conference on Learning Representations, ICLR 2015, May 7–9, 2015, Conference Track Proceedings, San Diego, CA, USA, 2015, <http://arxiv.org/abs/1412.6550>
- [43] L. Nair, Clip-Embed-KD: computationally efficient knowledge distillation using embeddings as teachers, CoRR [arXiv:2404.06170](https://arxiv.org/abs/2404.06170), 2024. <https://doi.org/10.48550/ARXIV.2404.06170>
- [44] G.E. Hinton, O. Vinyals, J. Dean, Distilling the knowledge in a neural network, CoRR [arXiv:1503.02531](https://arxiv.org/abs/1503.02531), 2015. <http://arxiv.org/abs/1503.02531>
- [45] M. Davis, G. Logemann, D.W. Loveland, A machine program for theorem-proving, Commun. ACM 5 (7) (1962) 394–397, <https://doi.org/10.1145/368273.368557>
- [46] S. Seitz, M. Alava, P. Orponen, Focused local search for random 3-satisfiability, CoRR [arXiv:cond-mat/0501707](https://arxiv.org/abs/cond-mat/0501707) 2005.
- [47] D. Vilenchik, It's all about the support: a new perspective on the satisfiability problem, J. Satisfiability Boolean Model. Comput. 3 (3–4) (2007) 125–139.